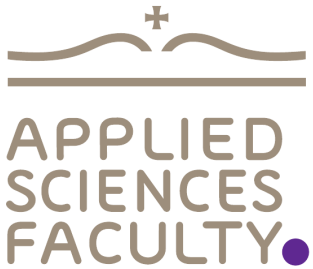
C++ — паралельні обчислення: потоки C++





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Потоки C++11

- В стандарті від 2011 року було (нарешті!) розширено стандартну бібліотеку засобами підтримки багатопоточності.
- Вона включає:
  - Явне формулювання вимог до моделі пам'яті C++.
  - Підтримка атомарних змінних.
  - Підтримка бар'єрів (fences).
  - Клас thread як об'єкт для керування потоком.
  - Мютекси.
  - Умовні змінні.
  - Більш високорівневі засоби, такі як future/promise, async.
- На жаль, деякі важливі абстракції, такі як thread pool, не ввійшли в цей стандарт.
- Їх поступово додають в наступних – C++14/17/20/23, є великі надії на C++26.

Пам’ять...

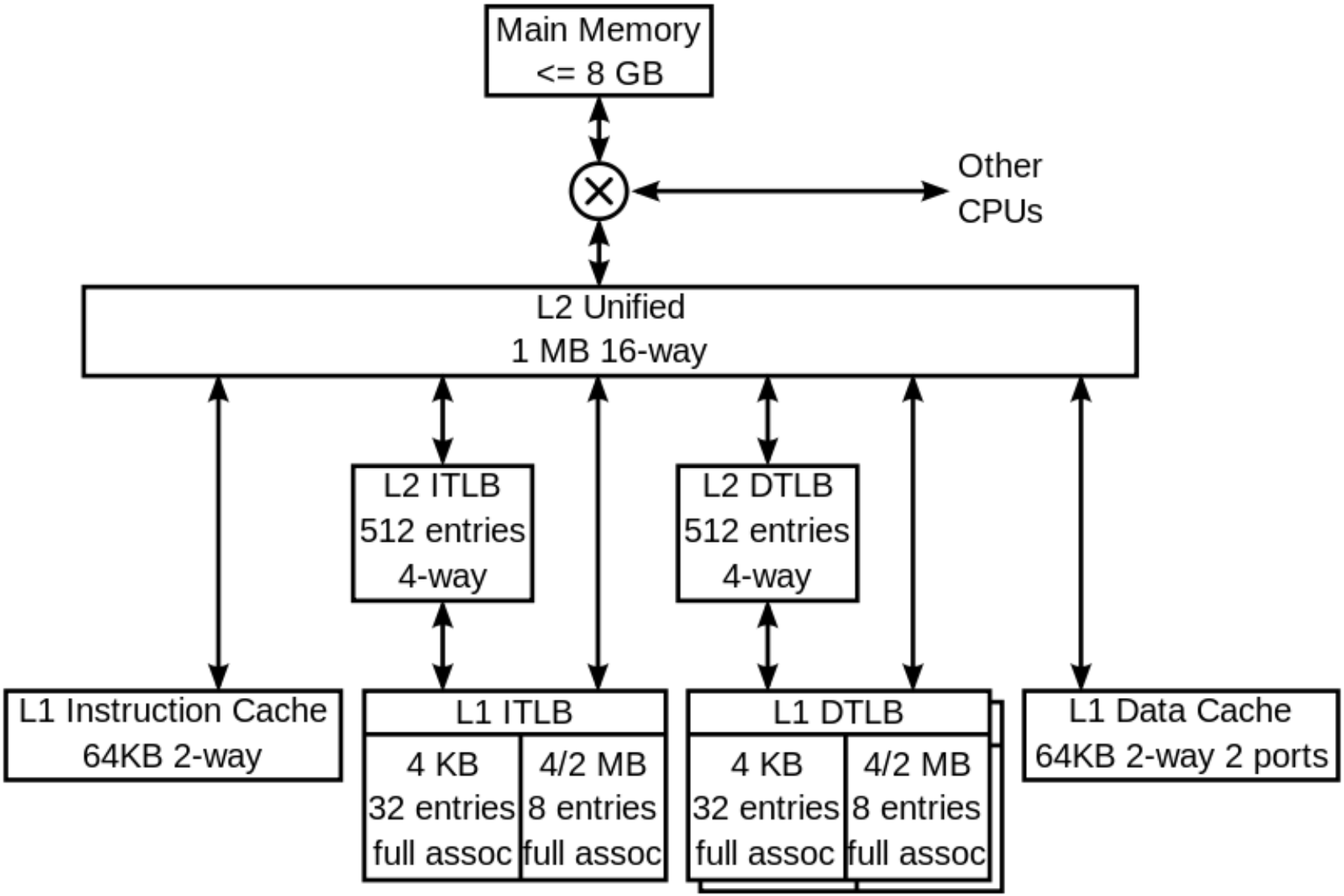
- Пам’ять, з точки зору сучасного процесора, надзвичайно складна річ. Так як швидкодія пам’яті в багато сотень раз менша за швидкодію власне процесора[1] і розрив цей росте, процесори використовують кеші. Кеш – це відносно невеликий об’єм швидкої пам’яті[2]. Так як об’єм і швидкодія (за фіксованих затрат) мало не обернено пропорційні, зазвичай використовується багато (два-три) кеші: L1, L2, іноді L3 — в такому порядку, все більші і все повільніші.
- Якщо система багатопроцесорна — а зараз такі складають дуже помітну частку (чи й всі сучасні, насправді), то така ієрархія буде для кожного процесора.
- Підтримка когерентності даних між ними – неймовірно складна задача. Інакше дуже легко виникатиме ситуація, коли та сама область пам’яті, відображена в кеші різних процесорів, має різні значення.
- Щоб не втратити всі виграші від наявності кількох процесорів (ядер), синхронізуючи пам’ять, йдуть на різноманітні компроміси.

[1] В тому сенсі, що, наприклад, прочитати два машинних слова займе стільки ж часу, скільки потрібно, щоб їх додати багато сотень раз.

[2] Все той же типовий інженерний конфлікт, тут – боротьба між ціною, місцем на кристалі та швидкодією.

Приклад

Архітектура кешу ядра K8, Athlon 64 CPU[3]:



[3] Взято із вікі, `https://en.wikipedia.org/wiki/CPU_cache`

Боротьба із false sharing, (C++17)

- Структурною одиницею кешу є **рядок**. На сучасних процесорах сімейства x86 його розмір – 64 байти.
- Якщо дві різних змінних потрапляють в один рядок кешу та змінюються із коду, що виконується на різних процесорах/ядрах, цей рядок буде постійно передаватися між цими процесорами – для збереження когерентності вмісту кешів. Це доволі повільно.
- Таке явище називається false sharing.
- Для боротьби із ним потрібно розташовувати змінні, що використовуються із різних потоків так, щоб вони не потрапляли в один рядок кешу[4].
- З іншого боку, змінні, які використовуються разом, варто тримати в одному рядку кешу – навіть якщо він мігруватиме між ядрами – потрібне true sharing.

[4] Мова, звичайно, про глобальні змінні – локальні "живуть" у стеку, і свої для кожного потоку та рознесені достатньо далеко.

- Щоб допомогти боротися із false sharing та сприяти true sharing, C++17 додав дві константи:

```
inline constexpr
  std :: size_t  hardware_destructive_interference_size  =  .....
inline constexpr
  std :: size_t  hardware_constructive_interference_size  =  .....
```

- Вони не менші за alignof(std::max_align_t).
- Перша – мінімальне зміщення між двома об’єктами, щоб не було false sharing.
- Друга – максимальний розмір, за якого ще буде true sharing.
- Фактично, це портабельний спосіб довідатися розмір рядка кешу першого рівня – L1.
- Див. також `https://devblogs.microsoft.com/oldnewthing/20230424-00/?p=108085` – “Why is std::hardware_destructive_interference_size a compile-time constant instead of a run-time value?”

Приклади

- ```
  struct keep_apart {
      alignas(std::hardware_destructive_interference_size)
          std::atomic<int> cat;
      alignas(std::hardware_destructive_interference_size)
          std::atomic<int> dog;
  };

  struct together {
      std::atomic<int> dog;
      int puppy;
  };
  struct kennel {
      // Other data members...
      alignas(sizeof(together)) together pack;
      // Other data members...
  };
  static_assert(sizeof(together) <=
                std::hardware_constructive_interference_size);
  ```
- Взято із `https://en.cppreference.com/w/cpp/thread/hardware_destructive_interference_size`.
  ```

Перевпорядкування інструкцій процесором

- Проблемою під час маніпуляції пам'яттю є не тільки архітектура кешу.
- Процесор, для оптимізації чи інших своїх потреб, має право перевпорядковувати команди (точніше, швидше навіть їх частини), якщо це не вплине на видимий результат. Однак, поняття видимого результату сильно відрізняється на однопроцесорній та багатопроцесорних системах.
- Детальніше див. `https://en.wikipedia.org/wiki/Superscalar_processor`, `https://en.wikipedia.org/wiki/Out-of-order_execution`, `https://en.wikipedia.org/wiki/Speculative_execution` та том 3 славнозвісного Агнера Фога: `http://www.agner.org/optimize/`.

Перевпорядкування інструкцій компілятором

- Компілятор теж має право переставляти команди!
- Уявимо код[5]:

```
int x;
bool x_ready = false; // По замовчуванню так і буде, для глобальних,
                      // але напишемо явно
void init (){
    x = initialize (); // # 1 Не використовує x_ready
    x_ready = true;    // # 2
}
```

- Компілятор цілком може переставити рядки # 1 і # 2 – з його точки зору вони ніяк не зв'язані.
- Однак, уявіть собі другий потік, який чекає, поки x_ready стане true, щоб почати використовувати x:

```
extern int x; extern bool x_ready;
void other_thread_task (){
    int y;
    while (!x_ready) // чекаємо на ініціалізацію
        this_thread :: sleep_for ( milliseconds {10});
    y=x;
     .....
```

- Та й у цьому потоці можуть бути проблеми — компілятор бачить, що x_ready в циклі не змінюється, і може винести читання із пам'яті за цикл.

[5] Див. Stroustrup B., "The C++ programming language ", Fourth edition.

Модель пам’яті C++11

- В стандарті C++11 нарешті було явно сказано, що доступ із різних потоків до різних ділянок в пам’яті не конфліктуватиме між собою.
- Раніше могла статися ситуація, коли дві незалежних змінних, скажімо, типу char, знаходячись фізично у одному машинному слові пам’яті, псувалися б під час одночасного звертання із різних потоків. Для її уникнення слід було консультуватися із документацією на компілятор, а то і його розробниками.
- Однак, про бітові поля (особливо такими, що знаходяться в одному і тому ж машинному слові) слід турбуватися самостійно! Поведінка за одночасного звертання буде невизначеною.
- З’явилися також засоби керування порядком доступу до пам’яті, про які далі, в розмові про atomic-змінні.

Клас std::thread

- Шматок роботи, який виконуватиметься паралельно, в термінах C++, називається **задача** (task).
- std::thread представляє собою інкапсуляцію системного потоку, котрий ту задачу виконуватиме.
- Задача представляється тим чи іншим callable — функцією (включаючи лямбда-функції) чи об'єктом-функцією.
- З моменту створення потік починає працювати — виконувати код своєї задачі[6].
- Його виконання відбуватиметься паралельно із тим потоком, який його запустив. Якщо останньому потрібно дочекатися завершення, він може використати метод join().

```
#include <iostream>
#include <thread>

void worker(){
    std::cout << "Hello from thread" << std::endl;
}
int main(){
    std::thread thread1(worker);
    std::cout << "Hello from main" << std::endl;
    thread1.join();
}
```

[6] Тобто, якщо потрібно створити потік, але дати йому роботу пізніше, доведеться якось потурбуватися про це, наприклад, створити задачу, яка почне працювати у відповідь на повідомлення, переслане іншим потоком.

- Іншою альтернативою є використати detach() — тоді потік продовжить виконання незалежно, не будучи прив'язаним до жодного об'єкту thread. Зазвичай це погана ідея.
- Перевірити, чи потік активний — чи з ним пов'язаний власне потік виконання, чи можна із ним робити join() (або detach()), можна методом joinable().
- Потік не буде joinable, якщо об'єкт thread сконструйовано по замовчуванню, вже було приєднано (join()) або від'єднано (detach()). Ще один важливий випадок — потік не може "приєднувати" сам себе. Потік, який вже завершився, але якого ще не було join() є joinable.
- Знищувати об'єкт thread потоку, що працює – погана ідея! Якщо об'єкт є joinable, деструктор викличе terminate() і завершить програму.
- **Зауваження**: вивід у cout із різних потоків гарантовано не зіпсує сам cout, але жодних гарантій про те, що і як виведеться, не дається. Запускаючи приклад із попереднього слайду це можна легко помітити.

- Якщо не може створити потік, кидає system_error з ”errc::resource_unavailable_try_again”. system_error є нащадком std::exception.
- Використання лямбда-функції:

```
int main(){
    std :: thread thread1(
        [](){ cout << ”Hello from thread” << endl; } );
    cout << ”Hello from main” << endl;
    thread1 . join ();
}
```

- Використання об’єкту-функції:

```
class Worker{
    std :: string text ;
public:
    Worker(std :: string text_=””): text ( text_ ) {}

    void operator ()() const {
        cout << text << endl;
    }
};
int main(){
    std :: thread thread1 { Worker(”Hello from thread”) };
    cout << ”Hello from main” << endl;
    thread1 . join ();
}
```

- Узагальнена ініціалізація тут обов’язкова — інакше (із круглими дужками), сприйме за оголошення функції.

Передача аргументів

- Callable, який працюватиме в потоці, можна передавати аргументи. Для цього існують декілька способів:
  - ◇ передати при створюванні потоку, тоді їх буде передано як аргументи під час виклику callable[7],
  - ◇ створити об'єкт-функцію, котрий зберігатиме все необхідне в собі або використати замикання.

```
void worker(const std::string& text ) { ..... }
struct arg_provider{
    const std::string& str;
    arg_provider(const std::string arg): str(arg){}
    void operator()(){ worker(str); }
};
    .....
        //! Using old-style constructor call
        std::thread thread1(worker, "Hello from thread 1:");
        //! Using unified initialization
        std::thread thread2{worker, "Hello from thread 2:"};
        //! Using closure:
        std::string ttext{"Hello from thread 3:"};
        std::thread thread3{ [&ttext](){ worker(ttext); } };
        //! Using object-function:
        std::thread thread4{ arg_provider("Hello from thread 4:") };
```

[7] За допомогою perfect forwarding – із викликаними цим обмеженнями

Передача аргументів з допомогою perfect forwarding

- Аргументи за значенням передаються без проблем – і не менш ефективно, ніж альтернативи.
- Однак, обмеження perfect forwarding спричиняють потребу їх враховувати:
  - Передавання аргументу за посиланням потребуватиме допомоги std::ref:

```
void update_str(std::string&);

.....
std::string arg = .....;
std::thread( update_str, std::ref(arg) );
```

Інакше буде помилка компіляції.

  - Аналогічно потрібно робити за передавання за константним посиланням – інакше може відбутися зайве копіювання:

```
void scan_str(const std::string&);

.....
std::string arg = .....;
std::thread( scan_str, std::cref(arg) );
```

- Перевантаження не працює, тому конкретне перевантаження доведеться обирати вручну і синтаксис буде потворним:

```
void fn(double);
void fn( std :: string );

.....
// std :: thread(fn, 5.3); // Помилка!
using cur_t_fn = void(*)(double);
std :: thread( static_cast <cur_t_fn>(&fn), 5.3 );
```

- Також, дуже важливим аспектом є час життя відповідних об'єктів – об'єкт повинен існувати весь час, коли він може використовуватися потоком.
- Приклад такої проблеми: "Why does my thread get a broken string as its initial parameter?".

Повернення результату

- Можна скористатися всім набором інструментів, котрі використовуються, якщо функція чи інший callable не може повернути результат безпосередньо — передавати вказівники чи посилання на місце, куди слід передати результат.
- Такий підхід широко розповсюджений, однак неелегантний. C++11 надає більш просунуті засоби – див. далі.

Інші засоби std::thread – 1

Типи:

- Вкладений клас id, котрий служить для представлення унікального ідентифікатора потоку[8]. Може бути ключем асоціативних контейнерів. Якщо потік завершився, значення його id може бути повторно використаним.
- `native_handle_type` — тип системного дескриптора (handle) потоку. Системозалежний.

Конструктори:

- Конструктор по замовчуванню – створює об'єкт, якому не відповідає жоден потік виконання.
- Конструктор-шаблон із змінною кількістю аргументів, який використовується для створення thread, пов'язаного із потоком виконання.
- Конструктор переміщення та оператор присвоєння із переміщенням. (Переміщення у об'єкт, який представляє потік виконання, викличе terminate().)
- Копіювання та присвоєння (без переміщення) — заборонені.

[8] Детальніше див. `http://en.cppreference.com/w/cpp/thread/thread/id`.

Інші засоби std::thread – 2

- join(), detach() та joinable() розглядалися раніше.
- Метод swap() та відповідна спеціалізація std::swap() поводяться звичним чином.
- `get_id()` повертає унікальний ідентифікатор потоку, типу std::thread::id. Для thread, що не представляють потік виконання та для від'єднаних потоків (зсередини), повертає id.
- `native_handle()` повертає системний дескриптор потоку, який можна передавати системним функціям (будь це, залежно від платформи, Pthread, winthreads чи щось інше). Дозволяє користуватися унікальними для поточного середовища засобами, не реалізованими (інкапсульованими) безпосередньо реалізацією потоків C++.
- Статична функція `hardware_concurrency()` повертає кількість потоків, для якої є апаратна підтримка. Системозалежна, але, зазвичай, це кількість логічних процесорних ядер.
- Готового способу припинити роботу потоку немає — це занадто системозалежна операція, тому слід потурбуватися самостійно.

Низькорівневі засоби підтримки використання std::thread, простір імен this_thread

- Функція `std::this_thread::get_id()` повертає id() поточного потоку.
- `std::this_thread::yield()` — рекомендує реалізації віддати залишок кванту потоку іншим потокам, що очікують на виконання.
- `std::this_thread::sleep_for()` — вказує потоку заснути на заданий інтервал часу. Приймає аргумент типу `std::chrono::duration`. Переведення системного годинника не впливатиме на тривалість (на відміну від sleep_until()). Через особливості диспетчеризації, може запуститися не миттєво після вказаного інтервалу.
- `std::this_thread::sleep_until()` — вказує потоку заснути до вказаного моменту. Приймає аргумент типу `std::chrono::time_point`. Враховує налаштування часу, тому, скажімо, якщо сказати очікувати півтори години, а потім зразу перевести годинник на годину, чекатиме півгодини. Через особливості диспетчеризації, може запуститися не миттєво після вказаного моменту.

TLS — Thread-Local Storage

- Глобальні змінні, змінні просторів імен, статичні члени класів[9] та локальні статичні змінні, оголошені із специфікатором `thread_local` є локальними для потоку — кожен потік матиме свою копію. Кожна така копія недоступна із інших потоків[10].
- Такі змінні ініціалізуються до першого використання, а їх деструктори викликаються при знищенні потоку. Однак, порядок конструювання не визначений.
- Одне із можливих використань — швидке[11] виправлення старого коду, який активно користувався статичними членами, наприклад, для збереження потенційно змінних значень по замовчуванню.
- Приклад – історично глобальна змінна errno тепер є thread local. Без цього не можна було б скористатися жодною функцією C в багатопоточному середовищі.
- В інших випадках потрібні рідко. Типовий виняток – створення локальних черг для потоків із thread pool.

[9] Які, як і локальні статичні змінні, фактичо є глобальними змінними із обмеженою або контрольованою (частково) ”областю” доступності.

[10] Хіба б останні необачно поділяться вказівником чи посиланням на такі TL-змінні.

[11] В сенсі quick-and-dirty.

TLS — Thread-Local Storage

- Детальніше про реалізацію в Linux: `https://chao-tic.github.io/blog/2018/12/25/tls` та `https://uclibc.org/docs/tls.pdf`. Під Windows: `http://www.nynaeve.net/?p=180` і далі за посиланням, `https://devblogs.microsoft.com/oldnewthing/20101122-00/?p=12233` та `https://docs.microsoft.com/en-us/windows/win32/procthread/thread-local-storage`.
- Ремарка: на x86 для звертань до TLS використовується сегментний регістр[12] GS (Windows та macOS) або FS (Linux[13]). На інших архітектурах можуть використовуватися спеціалізовані thread register.
  - GCC, за допомогою "Named Address Spaces", підтримує адресацію відносно них: __seg_fs, __seg_gs, наприклад: `__seg_fs int var = 1;`, за умови, що визначено відповідні макроси, __SEG_FS чи __SEG_GS.

[12] Для керування цими сегментними регістрами під x86-64, де сегментація не підтримується, використовується спеціальний MSR MSR_FS_BASE чи MSR_GS_BASE.

[13] Див. системний виклик arch_prctl().

- Бібліотека <chrono> розглядатиметься окремо, нижче лише декілька прикладів.
- Тривалість визначається шаблоном duration:

```
    //! Based on http://en.cppreference.com/w/cpp/thread/sleep_for
#if __cplusplus > 201103L
    using namespace std::chrono_literals; // C++14 only!
#endif
    std::cout << "Hello waiter" << std::endl;
    auto start = std::chrono::high_resolution_clock::now();
#if __cplusplus > 201103L
    std::this_thread::sleep_for(2s); //! C++14 only!
#else
    std::this_thread::sleep_for( std::chrono::seconds(2) );
#endif
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> elapsed = end-start;
    std::cout << "Waited " << elapsed.count() << " ms\n";
```

- Шаблон std::chrono::duration має два параметри — кількість одиниць часу та визначення одиниці, типу std::ratio — класу дробових чисел періоду компіляції.

- Приклад: `std::milli` це `std::ratio<1, 1000>`. Детальніше див. `http://en.cppreference.com/w/cpp/numeric/ratio/ratio`.
- Тоді, `std::chrono::milliseconds` це `duration<number, std::milli>`.
- У C++14 додали відповідний літерал, ms, і замість std::chrono::milliseconds(2000) чи взагалі `std::chrono::duration<long, std::ratio<1, 1000> >(2000)`, можна написати `2000ms`.
- C++20 додав: utc_clock, tai_clock (International Atomic Time), gps_clock, file_clock (той, що використовується для файлових систем).
- Детальніше див. Cppref:chrono::duration.
- "Using std::chrono".
- "What is the current time around the world? Utilizing std::chrono with time zones in C++23"
- ◇ "Around the World in C++: Exploring Time Zones with std::chrono".
- "There is a std::chrono::high_resolution_clock, but no low_resolution_clock" – більш точні таймери потребують більше ресурсів.
- "Exploring C++20 std::chrono - Calendar Types".
- ◇ "7 Practical std::chrono Calendar Examples (C++20)".
- "Time in C++: C++20 Brought Us Time Zones", "Time in C++: Creating Your Own Clocks with <chrono>" – зручна річ для тестів, симуляцій тощо, "Time in C++: Additional clocks in C++20".
- ◇ "The Portable Way to Extract Date and Time from a File's mtime in C++"

- Момент часу визначається шаблоном `std::chrono::time_point`. Його аргументи — годинник та тривалість, типу duration.
- C++11 підтримує три типи годинників: `system_clock`, `steady_clock`, `high_resolution_clock`[14].
- Приклад:

```
std::chrono::system_clock::time_point now = std::chrono::system_clock::now();
std::time_t now_c = std::chrono::system_clock::to_time_t(now
                                                  - std::chrono::hours(24));
std::cout << "24 hours ago, the time was "
          << std::put_time(std::localtime(&now_c), "%c %Z") << endl;

std::chrono::steady_clock::time_point start2 = std::chrono::steady_clock::now();
std::cout << "Hello World\n";
std::chrono::steady_clock::time_point end2 = std::chrono::steady_clock::now();
std::cout << "Printing took "
          << std::chrono::duration_cast<std::chrono::microseconds>(end2 - start2)
               .count()
          << "us.\n";
```

- Детальніше див. `http://en.cppreference.com/w/cpp/chrono/time_point` та `http://en.cppreference.com/w/cpp/chrono`.

[14] Увага – чи точність більша за інші, не гарантується, і він не обов'язково монотонний (steady).

jthread (C++20)

- std::jthread – аналог std::thread, який:
  - Виконує join() в деструкторі. (Явний join() підтримується).
  - Підтримує засоби добровільної (кооперативної) зупинки потоків[15].
- Станом на квітень 2020, підтримується лише GCC-10 із ключем -std=2a.

[15] Примусово потоки зупиняти/знищувати, фактично, неможливо. WinAPI, формально, містить відповідні засоби, TerminateThread, але вони мало придатні до використання. Див. `https://docs.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-terminatethread`.

std::stop_source та std::stop_token

- Алгоритм організації кооперативної зупинки потоків наступний:
  - Створити std::stop_source.
  - Отримати з нього std::stop_token, методом std::stop_source::get_token().
  - Передати цей std::stop_token потоку, який проситимемо зупинитися.
  - Запит на зупинку виконується методом std::stop_source::request_stop().
  - Те, що такий запит відбувся, можна перевірити за допомогою std::stop_token::stop_requested().
  - Перевіряти потрібно регулярно і достатньо часто!
- Підтримкою потокобезпечного спільного стану займається стандартна бібліотека.

Підтримка std::stop_source та std::stop_token в **std::jthread**

- Для реалізації можливості зупинки jthread містить внутрішній об’єкт типу std::stop_source.
- Якщо першим аргументом функції, що виконуватиметься в потоці, передати аргумент типу std::stop_token, функція через нього матиме доступ до цього спільного стану – jthread про це потурбується.
- Запит на зупинку здійснюється методом jthread::request_stop().
- Перевірити, чи це можливо, можна методом stop_token::stop_possible().
- Перевірити, чи був запит на зупинку: stop_token::stop_requested().

- Приклад:

```
std :: jthread   non_cooperating_thread ([](){
    for(int  i  =  0;  i<10;  ++i){
        std :: this_thread  :: sleep_for(1s);
        std :: cout  << "Rude thread   iter   :  " <<   i  << std :: endl;
    }
});

std :: jthread   cooperating_thread  ([]( std :: stop_token  token){
    for(int  i  =  0;  i<10 && !token.stop_requested ();  ++i){
        std :: this_thread  :: sleep_for(1s);
        std :: cout  << "Nice  thread   iter   :  " <<   i  << std :: endl;
    }
});

non_cooperating_thread . request_stop ();
cooperating_thread . request_stop ();
```

Підтримка кооперативної зупинки умовними змінними

- std::condition_variable_any доповнено методами, які прокидаються по request_stop():

```
template< class Lock, class Predicate >
bool wait( Lock& lock, std :: stop_token stoken, Predicate pred );
```

Аналогічно – для wait_for() та wait_until().

- Логіка їх наступна:

```
while (! stoken . stop_requested ()) {
    if (pred ()) return true;
    wait( lock );
}
return pred ();
```

std::stop_callback (draft)

- std::stop_callback дозволяє виконати функцію, коли до std::stop_source пов'язаного із переданою йому std::stop_token викликано request_stop().
- Приклад з `https://cor3ntin.github.io/posts/iouring/`:

```
std :: stop_source stop;
auto token = stop.get_token();
std :: stop_callback cb(token, [] {
    std :: puts("I don't want to stop at all\n");
});
std :: puts("Don't stop me now, I'm having such a good time\n");
stop.request_stop();
if (token.stop_requested()) {
    std :: puts("Alright\n");
}
```

- Виконується в потоці, який викликав request_stop(), або, якщо stop() було викликано до реєстрації callback – в потоці, що конструює std::stop_callback.

Операції синхронізації та доступ до пам'яті

- Операція синхронізації визначає, коли один потік бачить ефект якихось дій іншого потоку.
- Між операціями синхронізації компілятор та процесор мають право перевпорядковувати операції, лише б кінцевий результат був тим же.

Елементарні операції синхронізації поділяють на:

- Захоплення (acquire) — інші потоки побачать ефект цієї операції перш ніж ефект будь-якої наступної[16].
- Звільнення (release) — інші потоки побачать ефект всіх попередніх операцій, перш ніж цієї[17].
- Споживання (consume)[18] — більш м'який варіант захоплення. Ефекти, які не залежать від цієї операції, можуть відбутися перед нею.

Крім того можна говорити про:

- Узгоджену послідовність (sequential consistent), яка являє собою і acquire і release – повне впорядкування відносно операції.
- Розслаблений (relaxed) режим – без впорядкування.

[16] Прикладом операцій, що, по замовчуванню, гарантують таке впорядкування, є захоплення мютексу та читання із атомарної змінної.

[17] Прикладом операцій, що, по замовчуванню, гарантують таке впорядкування, є звільнення мютексу та запис у атомарну змінну.

[18] `https://devblogs.microsoft.com/oldnewthing/20230427-00/?p=108107` – "What's up with this new memory_order_consume memory order?".

- **Впорядкування атомарних операцій** керується спеціальним перерахуванням[19]:

```
enum memory_order {
    memory_order_relaxed,    memory_order_consume,
    memory_order_acquire,    memory_order_release,
    memory_order_acq_rel,        memory_order_seq_cst
};
```

- `memory_order_relaxed` — без впорядкування[20]. Гарантується лише атомарність операції. Приклад, до C++14 було дозволено навіть таке, C++14 заборонив:

```
// Thread 1:
r1 = y.load(std::memory_order_relaxed); // A
x.store(r1, std::memory_order_relaxed); // B
// Thread 2:
r2 = x.load(std::memory_order_relaxed); // C
y.store(42, std::memory_order_relaxed); // D
```

Цей код міг одночасно надати r1 == 42 і r2 == 42. Так, B йде після A, а D після C, однак, нічого не заважало (до C++14), щоб потік 1 побачив A після D, а потік 2 – C після B **одночасно**.

[19] `http://en.cppreference.com/w/cpp/atomic/memory_order`.

[20] Явно не режим для прикладного програмування, хоч і необхідний для написання глибоко-системного чи супер-оптимізованого коду.

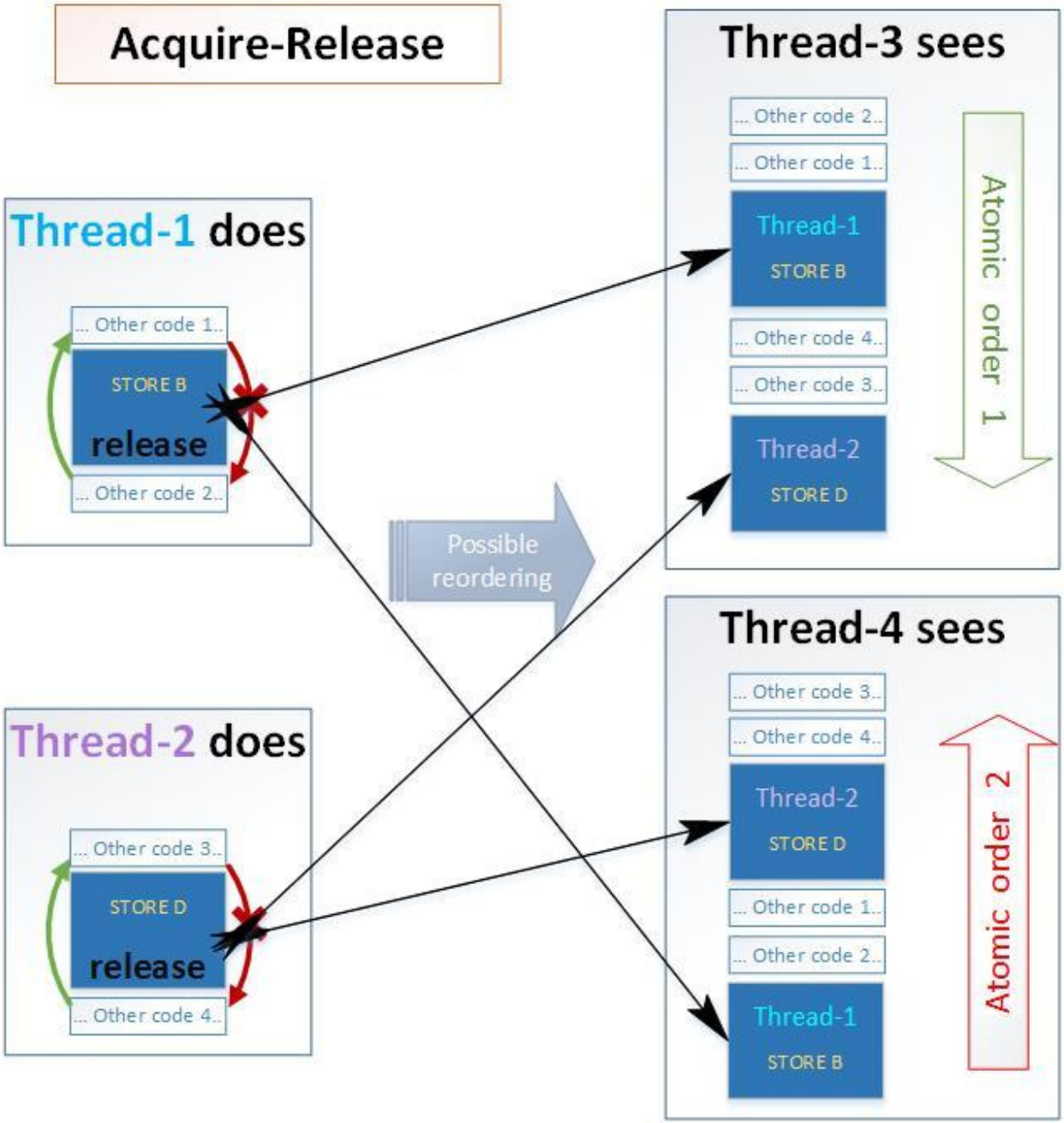
- `memory_order_release`, `memory_order_acq_rel`, `memory_order_seq_cst` — містять в собі операцію звільнення (release). Це власне збереження (запис — store), захоплення-запис-звільнення та повне впорядкування.
- `memory_order_consume` — читання (завантаження — load), здійснює операцію споживання (consume) стосовно змінних, залежних від цього завантаження, не впорядковуючи незалежні звертання.
- `memory_order_acquire`, `memory_order_acq_rel`, `memory_order_seq_cst` — містять в собі операцію захоплення (acquire). Це читання (load), захоплення-запис-звільнення, повне впорядкування.
- Іншими словами, `acq_rel` — і захоплення і звільнення, `seq_cst` — sequentially consistent, тобто, режим, коли кожен потік бачить результат всіх атомарних операцій у тому ж порядку, в якому вони виконані[21].

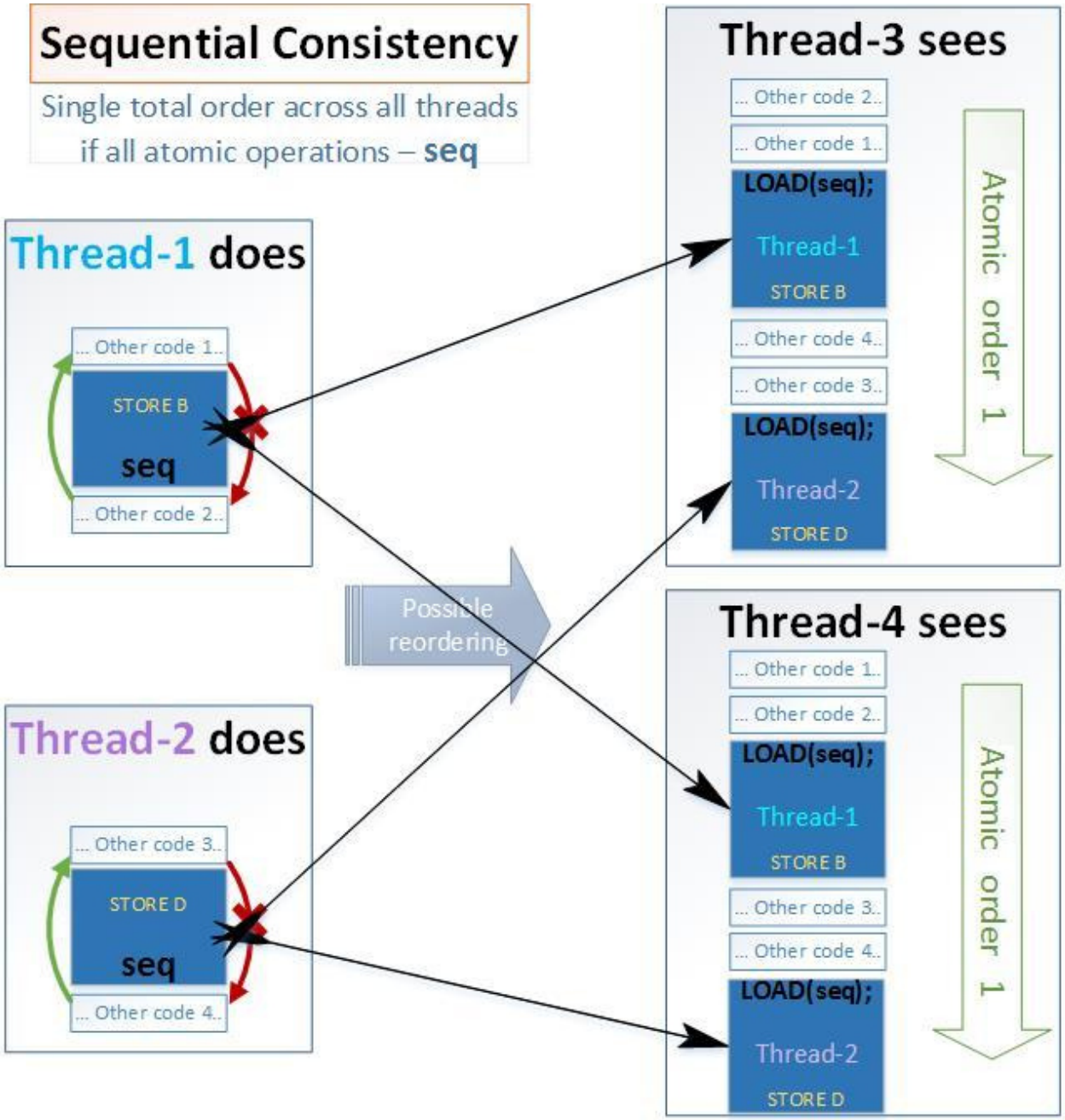
[21] Найбільш простий, але найменш продуктивний варіант. Звичайно, це стосується лише тих атомарних операцій, що використовують дане впорядкування.

- Різниця між `memory_order_acq_rel` та `memory_order_seq_cst` – перше впорядкування гарантує відсутність жодного перевпорядкування звертань до пам'яті відносно цієї операції, а друге додатково гарантує єдине впорядкування операцій між всіма потоками[22].
- `memory_order_acq_rel` стосується атомарних операцій виду Read-Modify-Write: compare_exchange_weak()/_strong(), exchange(), fetch_(add, sub, and, or, xor) та відповідних операторів.
- Також, std::mutex чи spinlock на базі atomic мають семантику `acq_rel`.
- Стаття із гарним оглядом моделей узгодженості C++ та того, як вони співвідносяться із моделями узгодженості апаратного забезпечення: `https://habr.com/en/post/197520/`, "Lock-free структуры данных. Основы: Модель памяти".
- ◇ Зокрема, автор стверджує, що (визнаний неактуальним) `memory_order_consume` був створений для уникнення перевпорядковування на Alpha для такого коду:

```
atomic<foo *> pFoo ;
// Set memory_order_consume  to fix
foo * p = pFoo.load( memory_order_relaxed );
int x = p->x;
```

[22] Див. `https://stackoverflow.com/questions/12340773/how-do-memory-order-seq-cst-and-memory-order-acq-rel-differ` та `https://en.cppreference.com/w/cpp/atomic/memory_order#Sequentially-consistent_ordering`.


Acquire-Release
Thread-1 does
... Other code 1..
STORE B
release
... Other code 2..
Thread-2 does
... Other code 3..
STORE D
release
... Other code 4..
Possible reordering
Thread-3 sees
... Other code 2..
... Other code 1..
Thread-1
STORE B
... Other code 4..
... Other code 3..
Thread-2
STORE D
Atomic order 1
Thread-4 sees
... Other code 3..
... Other code 4..
Thread-2
STORE D
... Other code 1..
... Other code 2..
Thread-1
STORE B
Atomic order 2


Sequential Consistency
Single total order across all threads
if all atomic operations – seq
Thread-1 does
... Other code 1..
STORE B
seq
... Other code 2..
Thread-2 does
... Other code 3..
STORE D
seq
... Other code 4..
Possible reordering
Thread-3 sees
... Other code 2..
... Other code 1..
LOAD(seq);
Thread-1
STORE B
... Other code 4..
... Other code 3..
LOAD(seq);
Thread-2
STORE D
Atomic order 1
Thread-4 sees
... Other code 1..
... Other code 2..
LOAD(seq);
Thread-1
STORE B
... Other code 3..
... Other code 4..
LOAD(seq);
Thread-2
STORE D
Atomic order 1

”Протягування” впорядкування

- Атрибут `[[carries_dependency]]` вказує, що release-consume залежність проходить скрізь функцію – компілятор не мусить вставляти додаткові бар’єри пам’яті.
- Функція `kill_dependency()` перериває таке розповсюдження залежностей.
- Детальніше див. `http://en.cppreference.com/w/cpp/atomic/kill_dependency` та `http://en.cppreference.com/w/cpp/language/attributes`.
- Стосується `memory_order_consume`, сучасні компілятори таким не заморочуються, ”піднімають” consume до acquire.

```
struct foo { int* a; int* b; };
std::atomic<struct foo*> foo_head[10];
int foo_array[10][10];

[[carries_dependency]] struct foo* fn1(int i) {
    return foo_head[i].load(memory_order_consume);
}

int g(int* x, int* y [[carries_dependency]]) {
    return std::kill_dependency(foo_array[*x][*y]);
}
```

Огляд atomic<>

- Абстракцією C++11 для атомарних змінних є шаблон atomic<>. Він, в залежності від платформи, використовує апаратні атомарні типи там, де це можливо, блокуючи явно для всіх інших.

  template< **class** T > **struct** atomic;
  template<> **struct** atomic< Integral >;
  template< **class** T > **struct** atomic<T*>;

- Кожне інстанціювання визначає атомарний тип.
- Спеціалізований для типів, котрі допускають апаратну реалізацію атомарних операцій.
- Вимога до типу, із яким інстанціюється — TriviallyCopyable[23],
- Впорядкуванням операцій можна керувати, передаючи параметр типу `std::memory_order`. По замовчуванню використовує `memory_order_seq_cst`.
- std::atomic не можна **ні копіювати, ні переміщати**.

[23] Тобто, має тривіальні конструктори копіювання, переміщення, операції присвоєння та присвоєння із переміщенням, деструктор і всі його члени та базові класи є TriviallyCopyable, віртуальних функцій чи базових класів немає. Конструктори тривіальні, якщо вони створені компілятором (не програмістом), при тому, клас не має віртуальних функцій, віртуальних базових класів, і конструктори всіх полів та предків тривіальні.

- Існують спеціалізації з іменами std::atomic_bool (atomic<bool>), atomic_int (atomic<int>) і т.д., майже для всіх інтегральних типів (включаючи типи виду `atomic_int_least8_t` та `atomic_int_fast8_t`, однак без типів виду int8_t).
- Існує спеціалізація для вказівників (див. далі).
- GCC потребує лінкувати із **libatomic**. (І достатньо свіжу версію – 4.8+).
- Дуже хороший відео-огляд: Fedor Pikus, ”C++ atomics, from basic to advanced. What do they really do?”, 2017. Презентація ”тут. Єдине, з того часу атомарні змінні стали трохи потужнішими, наприклад – підтримують множення.

Конструювання std::atomic<> – 1

- Конструктор із аргументом створює змінну із відповідним значенням:

```
atomic_int  a{5};                    atomic<int> b(11);
```

- Конструктор по замовчуванню, для сумісності із C, залишає об'єкт не ініціалізованим, за винятком звичної ініціалізації статичних та TLS-об'єктів. Після цього атомарну змінну слід "доініціалізувати" викликом `atomic_init()`:

```
atomic<int> c;
atomic_init(&c, 22);
```

- Навіть використання форми конструктора x{} вимагає виклику atomic_init – хоча самій величині буде присвоєно значення по замовчуванню, внутрішній стан власне атомарного об'єкта (блокування всілякі) все рівно не буде ініціалізовано.
- Повторні виклики atomic_init() чи виклики її до об'єкту, не сконструйованого по замовчуванню — undefined behavior.
- Увага, операція ініціалізації атомарною не є! (Хоча, зазвичай, це не проблема).

Конструювання std::atomic<> – 2

- Для статичної ініціалізації, можна скористатися макросом `ATOMIC_VAR_INIT`:

```
std :: atomic<int> d = ATOMIC_VAR_INIT(11);
```

Який належить до C11 і в C++, зазвичай, не потрібен.

Методи std::atomic<> — 1

- Оператор присвоєння, ”x = val;” та метод ”x.store(val);” — атомарно присвоюють значення. Val повинне бути типу type – аргументу шаблону atomic<>. Методу можна вказувати бажаний memory_order.
- Оператор присвоєння ”val = x;” та метод x.load() — атомарно читають (зберігають ”кудись”) значення. Методу load() можна передавати бажаний memory_order.
- Іншими словами, можна присвоювати як із значення відповідного типу так і у нього:

```
atomic<int> c{22};
c = 123;
int zz = c;
```

- Однак, на відміну від типового випадку, operator=() повертає копію а не посилання!

```
template< class T >
struct atomic{
    .....
    T operator=( T desired );
    atomic& operator=( const atomic& ) = delete;
```

- Самі атомарні змінні копіювати не можна.

Методи std::atomic<> — 2

- Метод `x.is_lock_free()` повертає, чи операції над даним типом здійснюватимуться без блокування.
- Атомарні типи можуть бути частково lock-free – лише для деяких операцій.
- З C++17, статична константа `is_always_lock_free` дозволяє перевірити, чи всі операції є атомарними.
- Для типів, що не підтримуються апаратно[24], **використовується явне блокування**.
- Взагалі, формально, будь-які атомарні типи можуть бути реалізовані із використанням блокувань, однак, всі основні реалізації надають lock-free спеціалізації для більшості або й всіх інтегральних типів, використовуючи підтримку, яку надають апаратні засоби.
- Метод x.exchange() атомарно записує нове значення та повертає попереднє: ”t2=x.exchange(t)”. Можна вказувати бажаний memory_order.

[24] Або воно не було використано реалізацією стандартної бібліотеки на даній платформі.

CAS для std::atomic<> — 1

- Методи `x.compare_exchange_weak()` та `x.compare_exchange_strong()` — порівнюють (побітово — як memcmp()) себе із вказаним об'єктом, якщо рівні — зберігає у себе передане значення, інакше – себе в передане значення (побітово – як memcpy()). Приклад:

```
atomic<int> val = 0;
// ...
int expected = val.load();    // Читаємо поточне значення
do {
    int next = calculate(expected);    // Рахуємо нове, наприклад, додаємо число.
} while (!val.compare_exchange_weak(expected,next));
// Записуємо нове або у val або у expected
// Рано чи пізно оновлене expected потрапить у val.
```

- Псевдокодом її роботу можна представити якось так[25]:

```
bool compare_exchange_strong(T& expected, T desired)
{
    if (this->load() == expected) {
        this->store(desired);
        return true;
    }
    else {
        expected = this->load();
        return false;
    }
}
```

Методи std::atomic<> — 3

- За тим стоїть наступна ідея: читається значення, із нього розраховується нове. Якщо значення із якого рахували, ще не змінилося (з іншого потоку) — записуємо оновлене назад. Інакше — беремо змінене, розраховуємо і знову намагаємося зберегти. Повторюємо, поки не вдасться[26].
- Використовується для реалізації типових при реалізації lock-free операцій CAS — compare-and-swap. (Але див. проблему ABA!)
- Різниця між `x.compare_exchange_weak()` і `x.compare_exchange_strong()` наступна — weak-варіант може іноді повертати false, навіть якщо val і expected рівні.
- Зроблено це не заради знущання — на деяких платформах реалізувати такий ”слабкий” варіант цієї функції простіше.
- Можна вказувати memory_order, в тому числі, окремо для успіху та окремо для невдачі.

[26] Але не забуваємо про livelock...

Методи std::atomic<> — 4

- Щодо різниці між weak і strong варіантами, див. також `http://en.cppreference.com/w/cpp/atomic/atomic/compare_exchange` та літературу по lock-free структурах.
- Причина spurious fail: `https://blogs.msdn.microsoft.com/oldnewthing/20180329-00/?p=98375` – наприклад, через те, що сучасні RISC процесори використовують підхід ”load locked/store conditional” чи таймаути NUMA.
- Спосіб вибору між weak та strong варіантом: `https://blogs.msdn.microsoft.com/oldnewthing/20180330-00/?p=98395`. Якщо коротко, коли відновлення після spurious fail недороге – підходить weak-варіант, якщо ж затратне або неможливе – потрібно скористатися strong-варіантом.
- Ще один огляд семантики атомних змінних: `https://habr.com/ru/post/517918/` – ”std::atomic. Модель памяти C++ в примерах”.

Методи std::atomic<> — 5

- Для інтегральних типів надається ряд арифметичних операцій, що виконуватимуться атомарно та відповідних перевантажених операторів:
  - `z=x.fetch_add(y[,order])` — `x.val += y;`
  - `z=x.fetch_sub(y[,order])` — `x.val -= y;`
  - `z=x.fetch_and(y[,order])` — `x.val &= y;`
  - `z=x.fetch_or(y[,order])` — `x.val |= y;`
  - `z=x.fetch_xor(y[,order])` — `x.val ^= y;`

  Всі повертають попереднє значення вмісту x, названий тут x.val. Всім можна вказувати опціональний параметр впорядкування доступу до пам'яті — order.
- fetch_add() та fetch_sub() та відповідні оператори для вказівників поводяться очікувано, завдяки спеціалізаціям.
- Також перевантажено наступні оператори: `++x`, `x++`, `--x`, `x--`, `x+=y`, `x-=y`, `x&=y`, `x|=y`, `x^=y`.
- Як і присвоєння, повертають копії а не посилання.

Методи std::atomic<> — 6

- Ілюстрація до попередніх пунктів – приклад атомарних операцій над інтегральними типами:

```
T operator+=( T arg );                  // Тільки для atomic<Integral>
T* operator+=( std::ptrdiff_t arg );    // Тільки для atomic<T*>

T fetch_and( T arg,                     // Тільки для atomic<Integral>
             std::memory_order order = std::memory_order_seq_cst );
T fetch_and( T arg,                     // Тільки для atomic<T*>
             std::memory_order order = std::memory_order_seq_cst ) vol
```

- До C++20 для floating-point типів такі операції не підтримувалися.
  - Тобто, `d += 1` не скомпілювалося б, якщо d має тип double.
  - `d = d + 1` скомпілюється, але **не буде атомарною**!
- C++20 додав fetch_add() та fetch_sub() операції.
- Їх поведінка стосовно заокруглення і т.д. може відрізнятися від такої для не-атомарних floating-point операцій, чи від описаного в std::numeric_limits, хоча стандарт і не рекомендує останнього.

Методи std::atomic<> та smart pointers – 7

- Стандарт C++11 включає ціле сімейство атомарних функцій для операцій над розумними вказівниками, `atomic_is_lock_free`, `atomic_store` і т.д. Детальніше див. `https://en.cppreference.com/w/cpp/memory/shared_ptr/atomic`.
- В C++20 їх замінено на атомарні спеціалізації[27], `std::atomic<std::shared_ptr<T>>` (до стандартизації – `atomic_shared_ptr`) та `std::atomic<std::weak_ptr<T>>` (`atomic_weak_ptr`), див. `https://en.cppreference.com/w/cpp/memory/shared_ptr/atomic2` та `https://en.cppreference.com/w/cpp/memory/weak_ptr/atomic2`.
- `https://travisdowns.github.io/blog/2020/07/06/concurrency-costs.html` – плата за використання тих чи інших конструкцій.
- `https://preshing.com/20140709/the-purpose-of-memory_order_consume-in-cpp11/` – крім назви, також список "плати" за acquire semantics.
- Див. також "Inside STL: The atomic shared_ptr", "Inside STL: Waiting for a std::atomic<std::shared_ptr<T» to change, part 1" – MSVC, "part 2" – libstd++ (GCC) та "On priority inversion in the use of a spinlock to ensure atomic access to a shared_ptr" від Раймонда Чена.
  - ◇ Також, він посилається на статтю "Implementing C++20 atomic waiting in libstdc++".

[27] А в C++26 – викинуто.

std::atomic_ref (C++20) DRAFT

- Шаблони атомарних посилань:

  template< **class** T > **struct** atomic_ref;
  template< **class** T > **struct** atomic_ref<T*>;

  можна використовувати із будь-яким TriviallyCopyable типом.
- На час існування такого об'єкту, об'єкт, на який він посилається, вважається атомарним.
- Звичайно, для цього потрібно, щоб всі потоки зверталися до нього лише за допомогою такого atomic_ref.
- Крім того, не можна, щоб atomic_ref посилалися на члени цього об'єкта.
- Спеціалізації для цілих типів, вказівників та floating point типів підтримують ті ж операції, включаючи арифметичні, що й std::atomic.
- Станом на квітень 2020, підтримуються лише GCC-10.
- `https://en.cppreference.com/w/cpp/atomic/atomic_ref`
- Огляд: `https://mariusbancila.ro/blog/2020/04/21/cpp20-atomic_ref/`.

Інтерфейс C для std::atomic<>

- Паралельно із атомарними змінними C++ розроблялися аналогічні для C, із тією ж семантикою. Відповідно:
- Для всіх методів є відповідні незалежні функції: atomic_store() atomic_load(), `atomic_compare_exchange_weak()`, `atomic_fetch_add()` і т.д.
- Варіант із явним вказанням впорядкування доступу до пам'яті: atomic_store_explicit(), аналогічно суфікс explicit додається і до інших операцій.[28],Детальніше див. `http://en.cppreference.com/w/cpp/atomic`.
- GCC потребує лінкувати із libatomic. (І достатньо свіжу версію – 4.8+).

[28] Через відсутність перевантаження функцій в C.

Double-checked locking

- Прості лічильники, типу тих, що використовуються розумними вказівниками, краще реалізовувати із використанням атомарних типів, ніж явного блокування.
- Якщо ініціалізація змінної повинна відбуватися ексклюзивно, може бути неефективним кожного разу блокувати, перш ніж перевірити прапорець, який вказує чи змінна ініціалізована. В таких ситуаціях використовується ідіома double-checked locking: спершу перевіряється прапорець, якщо він false – робиться спроба блокування:

```
C x; // Ініціалізація має відбуватися ексклюзивно
mutex lx; // Мьютекс для ініціалізації x
atomic<bool> x_init {false}; // Атомарна змінна, true, якщо
                             // x ініціалізовано, інакше -- false
void fn(){
    if (!x_init) { // Якщо не ініціалізовано:
        lx.lock();
        if (!x_init) { // Якщо ВСЕ ЩЕ неініціалізовано
            // ... ініціалізація ...
            x_init = true;
        }
        lx.unlock();
    }
// Використання x
}
```

call_once() та once_flag

- Увага: якби в попередньому прикладі x_init не була атомарною — компілятор міг би перемістити запис у неї!
- Ідіома double-checked locking достатньо поширена, щоб стандартна бібліотека C++11 мала її явну підтримку – функцію call_once() та тип once_flag (із <mutex>). Попередній код, із їх використанням, можна переписати так:

```
C x; // Ініціалізація має відбуватися ексклюзивно
once_flag x_init; // Прапорець
C init_x_fn(int v) { ..... } // Функція ініціалізації

void fn(){
    call_once( x_init , init_x_fn , 5);
    // Зверніть увагу, як передається аргумент функції ініціалізації.
    // Використання x
}
```

- Гарний огляд: `https://quuxplusone.github.io/blog/2020/10/23/once-flag/`.
- Ідіома вважається неприйнятною в Java, оскільки потребує знання про memory consistency процесора, на якому виконується (ймовірно, лише через Alpha AXP);

Атомарні прапорці

- Типу `atomic_flag` — найпростіший атомарний тип, єдиний, для якого стандартом гарантується lock-free. Хоча, всі адекватні реалізації надають також lock-free спеціалізації atomic<> для, принаймні, цілих типів.
- Представляє собою один біт інформації. **Дуже низькорівневий засіб**!
- Має конструктор по замовчуванню (не ініціалізує, для сумісності із C), виклик конструктора t чи присвоєння йому ATOMIC_FLAG_INIT, ініціалізує значенням 0 (false).
- Увага, ATOMIC_FLAG_INIT можна користуватися лише так:

  ```
  atomic_flag  flag  = ATOMIC_FLAG_INIT;
  ```

  Ефект в інших контекстах (передача конструктору) — не визначений. Зате його безпечно використовувати для конструювання статичних об'єктів.
- Як і інші атомарні типи, не можна копіювати чи переміщати.
- Можна очистити: fl.clear([order]), із необов'язковим параметром order типу memory_order.
- Можна встановити: "old = fl.test_and_set([order]);" – повертає старе значення fl, встановлює fl в true.

Інтерфейс C для атомарних прапорців

- Існують відповідні незалежні функції, atomic_XXX() і atomic_XXX_explicit() – інтерфейс C: `atomic_flag_test_and_set()`, `atomic_flag_test_and_set_explicit()`, `atomic_flag_clear()`, `atomic_flag_clear_explicit()`.
- Детальніше див. `http://en.cppreference.com/w/cpp/atomic/atomic_flag`.

Spinlock із використанням atomic_flag

УВАГА Це лише демонстрація принципу! Реалізований таким чином spinlock доволі неефективний.

```
std :: atomic_flag  lock  = ATOMIC_FLAG_INIT;
      ......................
// Пробуємо захопити блокування
while (lock. test_and_set (std :: memory_order_acquire))
     {}; // Якщо не вдалося -- повторюємо у циклі

// Робота із ексклюзивним доступом

lock. clear (std :: memory_order_release);    // Розблоковуємо
```

Або, загорнувши у клас:

```
class  spin_mutex {
     atomic_flag  flag  = ATOMIC_FLAG_INIT;
public:
     void  lock()  { while( flag . test_and_set (std :: memory_order_acquire)); }
     void  unlock()  { flag . clear (); }
};
```

- Обґрунтування, що тут немає загрози deadlock: https://www.justsoftwaresolutions.co.uk/cplusplus/can

Spinlock із використанням atomic_flag – 2

- Більш повний та дещо оптимізований варіант, взято тут: `https://habr.com/ru/post/328362/`.

```
class spinlock_t {
    std :: atomic_flag lock_flag ;
public:
    spinlock_t () { lock_flag . clear (); }

    bool try_lock () {
        return ! lock_flag . test_and_set ( std :: memory_order_acquire);
    }
    void lock () {
        for ( size_t i = 0; ! try_lock (); ++i)
                    if (i % 100 == 0) std :: this_thread :: yield ();
    }
    void unlock () {
        lock_flag . clear ( std :: memory_order_release);
    }
};
```

Бар'єри пам'яті — fences

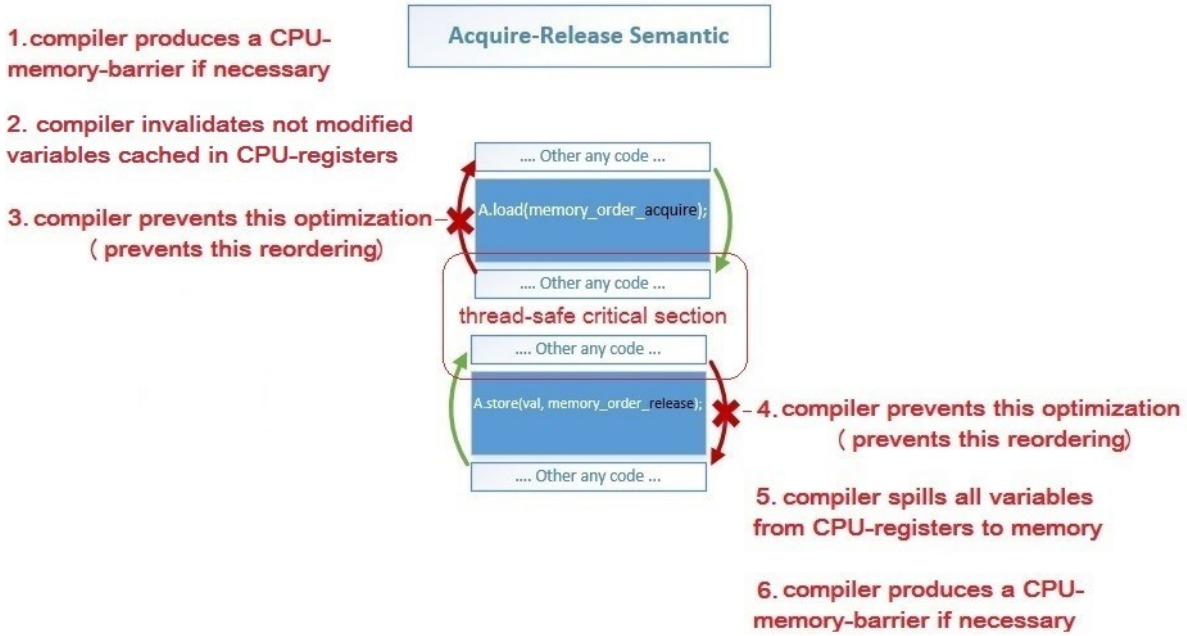
- Надзвичайно низькорівневі конструкції!
- Дозволяють вказати впорядкування доступу до пам'яті для неатомарних та релаксованих атомарних операцій.
- **extern** "C" **void** atomic_thread_fence(std :: memory_order order);

  //! Відрізняється від попереднього тим, що не вставляє
  //! машинних команд для синхронізації
  //! Лише впливає на генерацію коду компілятором
  //! Використовується для синхронізації коду в потоці та
  //! обробника сигналів цього ж потоку.
  extern "C" **void** atomic_signal_fence(std :: memory_order order);

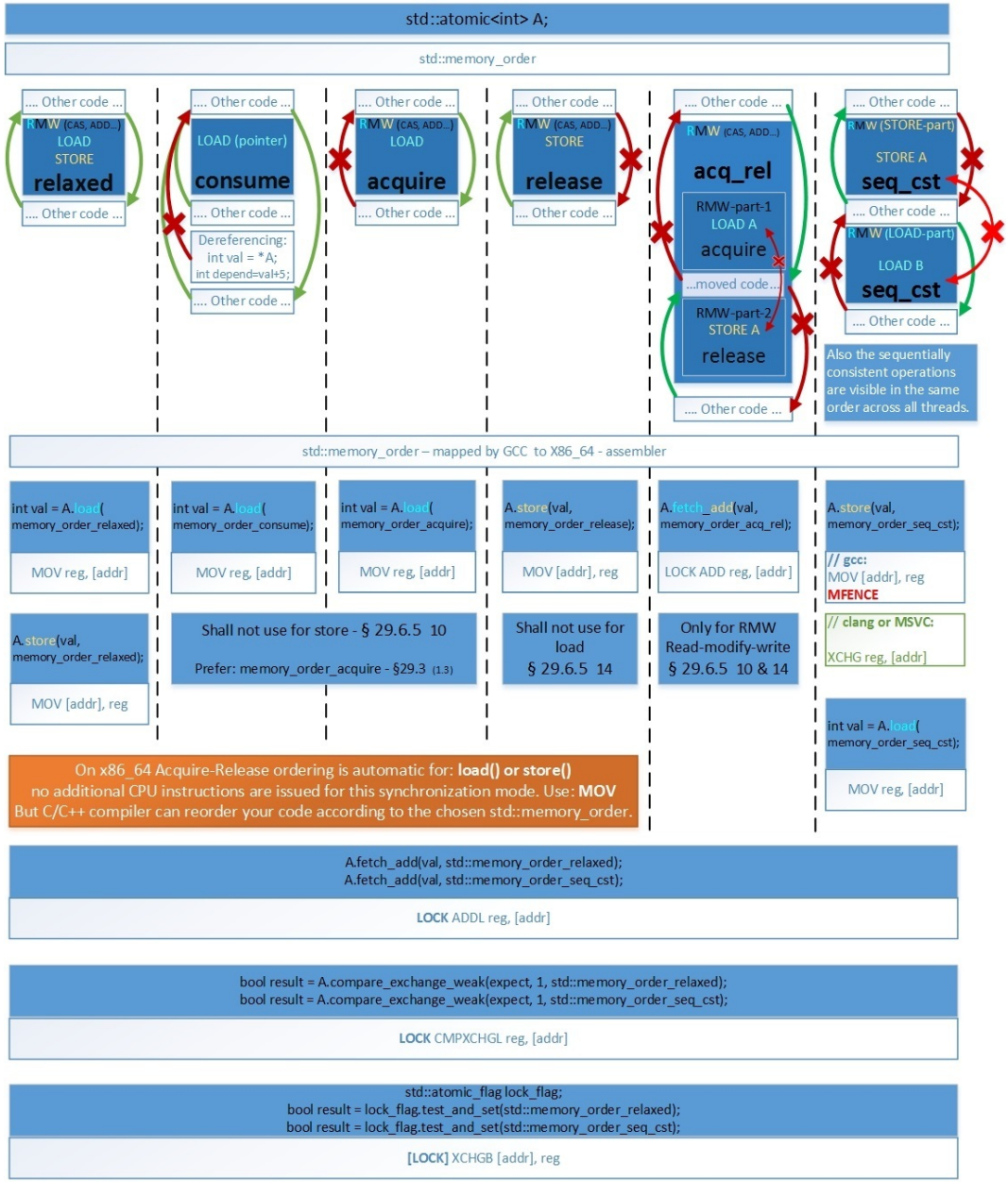
- Подробиці див. `http://en.cppreference.com/w/cpp/atomic/atomic_thread_fence` та `http://en.cppreference.com/w/cpp/atomic/atomic_signal_fence`.
- Особливості функціонування: "Acquire and Release Fences Don't Work the Way You'd Expect".
- ◇ Зокрема, release для fence не є (лише) release-операцією – він також гарантує, що попередні відносно нього операції не перемістяться після наступних записів (записи під бар'єром не піднімуться вище за нього). Атомарний запис з семантикою release забороняє пересування лише операцій вище відносно себе.

atomic_thread_fence

- Бар'єр забороняє компілятору виконувати перевпорядкування, що суперечать вибраній узгодженості. Наприклад, для seq_cst буде заборонено перенесення взагалі, а при acquire – лише вверх, перенесення вниз по коду можуть виконуватися.
- Компілятор вивантажує в пам'ять змінні, присутні в регістрах (spilling) перед release, щоб завантажити їх потім, після acquire.
- За потреби, він також вставляє в код інструкції-бар'єри відповідної архітектури (mfence на x86, dmb для ARMv7, див. також `https://www.cl.cam.ac.uk/~pes20/cpp/cpp0xmappings.html` – зауважте, архітектура x86 дуже строга по відношенню до впорядкування звертань в пам'ять, більшість операцій машинних команд-бар'єрів не потребують).



- Memory consistency і x86, взято тут `https://habr.com/ru/post/328362/`



Оптимізації з врахуванням atomic

- Базова ідея оптимізації атомарних змінних – компілятор може робити операції **більш** атомарними, ніж в коді – доти, поки це не заважатиме вимозі можливості прогресу виконання (”forward progress requirements”).
  - Наприклад, він може замінити (приклад з посилання далі):

```
void inc(std::atomic<int> *y) {
  *y += 1;
}
std::atomic<int> x;
void two() {
  inc(&x);
  inc(&x);
}
```

на:

```
std::atomic<int> x;
void two() {
  x += 2;
}
```

  - Інший приклад – пропагування констант, скажімо, перетворення `atomic<T>::fetch_and(~(T)0)` в `atomic<T>::load()` тощо.

Мютекси

- Стандартна бібліотека C++11 надає засоби явної синхронізації. Поміж них найважливішими є мютекси. Слово походить від скорочення словосполучення MUTual EXclusion — ”взаємне виключення”.
- Мютекси дозволяють організувати ексклюзивний (серіалізований) та синхронний доступ до спільних змінних, боротися із Data race.
- Однак, самі по собі вони нічого не захищають — це лише засіб, використання якого базується на домовленостях — жоден потік не звертатиметься до даних, не захопивши (не заблокувавши) відповідний мютекс. Реалізація мютекса лише забезпечує, щоб ніколи його не могли захопити два потоки одночасно[29].
- Типове використання – перш ніж спробувати доступитися до спільного ресурсу, потік пробує заблокувати (захопити) мютекс. Якщо вдалося — значить є ексклюзивний доступ, можна працювати. Завершивши, слід розблокувати (звільнити) мютекс, щоб інші потоки могли продовжити роботу. Якщо ж захопити не вдалося — потік блокується, поки мютекс не буде звільнено. Є різноманітні вдосконалення, які буде розглянуто далі.

[29] Для їх реалізації, звичайно, потрібні апаратні атомарні змінні чи хоча б прапорці.

- Найпростішим із них є клас std::mutex. Більш просунуті, але трішки менш ефективні модифікації: std::timed_mutex, std::recursive_mutex, std::recursive_timed_mutex, std::shared_timed_mutex (C++14).
- Почнемо із найпростішого — std::mutex.
- Конструктор по замовчуванню створює його не захопленим.
- Знищення захопленого мютекса — undefined behavior.
- Завершення потоку, який володіє мютексом — undefined behavior.
- Мютекс не можна ні копіювати, ні переміщати.
- Метод m.lock() захоплює мютекс. Якщо мютекс заблоковано, не завершується, поки не вдасться захопити. Не повертає нічого.
- Метод m.try_lock() намагається захопити мютекс. В будь-якому випадку, повернення відбувається зразу – потік не блокується (не зупиняється). Якщо вдалося захопити — повертає true, інакше — false. Можна використовувати, коли потік має іншу роботу, якщо захопити не вдалося. Формально, з його допомогою можна реалізувати примітив spinlock, але це дуже неефективно, див.: `http://stackoverflow.com/questions/35331209/spinlock-vs-stdmutextry-lock`.
- **Увага!** Через особливості реалізації, на деяких платформах може повернути false, навіть коли мютекс ніким не заблокований. При цьому слід просто повторити спробу.
- Метод m.unlock() звільняє (розблоковує) мютекс.

- Може надавати[30] доступ до дескриптора системного засобу, використаного для реалізації цього мютекса:

```
native_handle_type   std :: mutex:: native_handle ();
```

- Детектує певні види проблем та кидає виключення std::system_error. При тому не блокується.
- Оголошені, в основному, <mutex>. Деякі – в <shared_mutex> (C++14).

[30] Може не надавати – на розсуд реалізації. Причина в тому, що не у всіх випадках на всіх платформах мютексу відповідатиме якийсь дескриптор

Деякі можливі коди помилок м'ютексів

- resource_deadlock_would_occur — відбувся б deadlock. Але виявляє лише найпростіші ситуації, навіть deadlock виду:

```
try{
    //! Deadlock on many implementations, exception on some...
    m.lock();
    m.lock();
}
catch (system_error& e) {
    m.unlock();
    cout << e.what() << endl << e.code() << endl;
}
```

  заблокувався під GCC-4.8/4.9/5.2, clang-7/8/9, Windows/Linux. Мовчки завершилося GCC-7.5/8.3/9.2, Windows/Linux. А ось MSVS 2015/2017/2019 виявила проблему і повідомила: "device or resource busy".
- resource_unavailable_try_again — системний дескриптор недоступний.
- device_or_resource_busy — системний дескриптор вже захоплено.
- invalid_argument — передано невірний системний дескриптор конструктору.
- operation_not_permitted — потоку заборонено виконувати вказану операцію.

Приклад:

```
void worker(int idt, std::mutex& m){
    for(int i = 0; i<repeats; ++i ){
        m.lock();
        cout << "Hello from thread-" << idt << ": " << i << endl;
        m.unlock();
    }
}

    ..........................
    int Nthreads = 10;
    vector<std::thread> threads;
    std::mutex m;
    for(int i = 0; i<Nthreads; ++i){
        threads.emplace_back( worker, i, std::ref(m) );
    }
    for(int i = 0; i<repeats; ++i ){
        m.lock();
        cout << "Hello from main: " << i << endl;
        m.unlock();
    }
    for(auto& thread : threads)
        thread.join();
```

Коментарі до прикладу та RAII-обгортки

- Вивід програми буде тепер значно більш впорядкованим.
- Зауважте, що блокуються мінімальні необхідні блоки — якщо, наприклад, захоплювати мютекс на початку функції worker(), паралелізму, по суті, не буде взагалі — потоки працюватимуть послідовно.
- Якщо в процесі роботи коду між lock() і unlock() станеться виключення, мютекс залишиться заблокованим — відбудеться deadlock.
- Звичайно, можна якось викрутитися — скористатися try-catch(...) і там звільнити. Але такі рішення громіздкі та вразливі до помилок.
- Більш зручним буде скористатися ідіомою RAII.
- Стандартна бібліотека надає відповідні класи: std::lock_guard, std::unique_lock, std::shared_lock (C++14).
- Найпростіший із них — std::lock_guard. Його конструктору передається мютекс. Він захоплюється в конструкторі, звільняється в деструкторі. Тепер навіть виникнення виключення не залишить мютекс заблокованим.
- std::lock_guard не можна ні копіювати ні переміщати.
- Решта класів цього сімейства буде розглянуто в подальшому.

Попередній приклад із використанням std::lock_guard

```
void worker(int idt, std::mutex& m) {
    for(int i = 0; i<repeats; ++i ) {
        std::lock_guard<std::mutex> lock(m);
        cout << "Hello from thread-" << idt << ": " << i << endl;
    }
}

        ..........................
        int Nthreads = 10;   vector<std::thread> threads;  std::mutex m;
        for(int i = 0; i<Nthreads; ++i)
            threads.emplace_back( worker, i, std::ref(m) );
        for(int i = 0; i<repeats; ++i ){
            std::lock_guard<std::mutex> lock(m);
            cout << "Hello from main: " << i << endl;
        }
        for(auto& thread : threads)                thread.join();
```

- Рекомендується надавати перевагу std::lock_guard та аналогам над безпосередньою маніпуляцією мютексами!
- Однак, це не привід захоплювати мютекс на довший період, ніж мінімально необхідний! На крайній випадок — використайте вкладений блок.

std::recursive_mutex

- Повторне рекурсивне захоплення std::mutex приведе до deadlock.
- Стандартна бібліотека надає клас std::recursive_mutex, який можна повторно захоплювати довільну кількість раз.
- Перш ніж бути звільненим ”повністю” — надати можливість іншим потокам захопити його, він повинен бути звільненим ту ж кількість раз, що був захопленим.
- У всьому решта — еквівалентний std::mutex, хоч і трохи повільніший, зокрема, через необхідність підрахунку – скільки раз захоплено[31].
- Максимальна глибина вкладеності — на розсуд реалізації. Якщо вона буде досягнута, методи, які нічого не повертають (lock()) кинуть виключення std::system_error із кодом, методи, що повертають bool (try_lock()) — повернуть false.

[31] На прикладі легких ”замків” на читання-запис Windows, показано, де можуть бути інші накладні витрати на рекурсивний м’ютекс: `https://devblogs.microsoft.com/oldnewthing/20220304-00/?p=106309`, ”If the slim reader/writer lock (SRWLOCK) doesn’t remember who the shared lock owner is, does that mean it’s okay to acquire it recursively?” – реалізація не є повністю fair, щоб давати шанс потокам, що планують записувати, але при тому не веде облік рекурсивного виклику.

std::timed_mutex і std::recursive_timed_mutex

- Між lock() — чекати довічно та try_lock() — не чекати взагалі, є безліч проміжних варіантів — чекати певний інтервал часу або до певного моменту.
- std::timed_mutex та std::recursive_timed_mutex надають реалізацію такому блокуванню з очікуванням. (Рекурсивний варіант дозволяє рекурсивні захоплення тим же потоком.)
- Вони, крім методів, доступних у std::mutex, мають:

```
template< class Rep, class Period >
bool try_lock_for ( const std :: chrono :: duration <Rep,Period>&
                                        timeout_duration );
template< class Clock, class Duration >
bool try_lock_until ( const std :: chrono :: time_point<Clock,Duration>&
                                        timeout_time );
```

- **Увага!** Як і try_lock(), через особливості реалізації, на деяких платформах може повернути false, навіть коли мютекс ніким не заблокований. При цьому слід просто повторити спробу.
- Якщо інтервал часу менший-рівний за нуль, або момент часу вже минув, поводяться як try_lock().

- Максимальна глибина вкладеності рекурсивного — на розсуд реалізації. Якщо вона буде досягнута, методи, які нічого не повертають (lock()) кинуть виключення std::system_error із кодом, методи, що повертають bool (try_lock(), try_lock_for(), try_lock_until()) — повернуть false.

std::shared_timed_mutex (C++14)

- Цей мютекс, на відміну від інших, допускає два варіанти володіння:
- shared — кілька потоків можуть одночасно володіти цим мютексом,
- exclusive — лише один потік може володіти ним в такому режимі.
- Використовується у випадках, коли кілька потоків можуть одночасно читати, але писати може лише один. Якщо хоча б один потік читає, запис буде заблоковано, поки відбувається запис – блокуватимуться всі читання.
- Методи, присутні і в інших мютексів: lock(), try_lock(), try_lock_for(), try_lock_until(), unlock() — захоплюють/звільняють в ексклюзивному режимі.
- Для спільного (shared) захоплення, є їх аналоги: lock_shared(), try_lock_shared(), try_lock_shared_for(), try_lock_shared_until(), unlock_shared().
- Поведінка та ж, що і в аналогів. Максимальна кількість потоків, що спільно використовують цей мютекс системозалежна, однак гарантується, що більша за 10 000. Якщо максимуму досягнуто — відповідні методи не повертаються, поки їх не поменшає.
- Оголошений в <shared_mutex>.

std::shared_mutex (C++17)

- C++17 додав ще спрощений варіант – shared_mutex.
- Оголошений в <shared_mutex>.
- А ось рекурсивного варіанту такого RW-м'ютекса немає.

std::lock_guard

- std::lock_guard — дуже простий та ефективний клас. Накладні витрати відсутні.
- Однак, він не дозволяє виконувати над "своїм" мютексом ніяких операцій.
- Реалізація із GCC:

```
template <class _Mutex>
class lock_guard { // class with destructor that unlocks a mutex
public:
    using mutex_type = _Mutex;
    explicit lock_guard(_Mutex& _Mtx) : _MyMutex(_Mtx) {
        _MyMutex.lock();
    }
    lock_guard(_Mutex& _Mtx, adopt_lock_t) : _MyMutex(_Mtx) { }
    ~lock_guard() noexcept {
        _MyMutex.unlock();
    }
    lock_guard(const lock_guard&) = delete;
    lock_guard& operator=(const lock_guard&) = delete;
private:
    _Mutex& _MyMutex;
};
```

std::unique_lock — більш просунутий варіант RAII-обгортки

- std::unique_lock — більш потужний, дозволяє виконувати над мютексом, яким володіє, всі його операції. При тому, не звільнений м'ютекс буде звільнено в деструкторі.
- Дозволяє передачу володіння — має конструктор переміщення та оператор переміщення із присвоєнням.
- Має метод swap() та спеціалізацію функції std::swap().
- Звичайно, доступні лише ті операції, які підтримує сам мютекс.
- ◇ Скажімо, ні методу try_lock_for(), ні відповідного конструктора (див. далі) не буде в спеціалізації для std::mutex, std::unique_lock<std::mutex>.

Конструктори std::unique_lock

- Множина конструкторів виражає різноманіття операцій над мютексами, якими може володіти цей RAII-клас[32]:

```
unique_lock();                                                    (1)
unique_lock( unique_lock&& other );                               (2)
explicit unique_lock( mutex_type& m );                            (3)
unique_lock( mutex_type& m, std::defer_lock_t t );                (4)
unique_lock( mutex_type& m, std::try_to_lock_t t );               (5)
unique_lock( mutex_type& m, std::adopt_lock_t t );                (6)
template< class Rep, class Period >
unique_lock( mutex_type& m,
             const std::chrono::duration<Rep,Period>&
                   timeout_duration );                            (7)
template< class Clock, class Duration >
unique_lock( mutex_type& m,
             const std::chrono::time_point<Clock,Duration>&
                   timeout_time );                                (8)
```

[32] Список взято тут: `http://en.cppreference.com/w/cpp/thread/unique_lock/unique_lock`.

- Типи-теги std::defer_lock_t, std::try_to_lock_t та std::adopt_lock_t та відповідні константи (для автовиведення): std::defer_lock, std::try_to_lock та std::adopt_lock, заслуговують окремої уваги.
- std::defer_lock_t вказує не захоплювати володіння мютексом (відкласти це на потім).
- std::try_to_lock_t — спробувати захопити мютекс, але без блокування.
- std::adopt_lock_t — вважати, що потік вже володіє мютексом, який асоціюється із unique_lock.
- Рекурсивне захоплення можливе лише для рекурсивних мютексів. Для решти це undefined behaviour.
- Метод owns_lock() дозволяє перевірити, чи об’єкт володіє м’ютексом.
- ◇ Він потрібен, тому що конструктори не повертають значень, у випадку використання конструктора з std::try_to_lock.

Методи std::unique_lock

- Методи lock(), try_lock(), try_lock_for(), try_lock_until(), unlock() поводяться як відповідні методи пов'язаного мютекса.
- `mutex_type* release();` — розриває зв'язок між unique_lock і мютексом, повертає вказівник на мютекс, або нульовий вказівник, якщо нічого не було асоційовано. Нічого не розблоковує і т.д — лише забирає контроль у unique_lock.
- `mutex_type* mutex() const;` — повертає вказівник на асоційований мютекс (або нульовий вказівник).
- `bool owns_lock() const;` — повертає true, якщо є асоційований мютекс і його захоплено.
- `explicit operator bool() const;` — ефективно еквівалентне owns_lock.

std::shared_lock (C++14)

- Аналог std::unique_lock для мютексів спільного використання (shared_timed_mutex).
- Потрібен для RAII-маніпуляції ними у спільному режимі (для ексклюзивного достатньо shared_lock чи lock_guard).
- Методи lock(), try_lock(), try_lock_for(), try_lock_until(), unlock() викликають відповідні shared_lock() і т.д.
- Оголошено в <shared_mutex>.
- Ймовірно, був доданий до стандарту через проблеми із додаванням нових методів до std::unique_lock – через все ту ж зворотну сумісність.

Алгоритми std::lock та std::try_lock

- Задача захопити декілька ресурсів для однієї операції виникає часто. Однак, при тому зростає загроза deadlock-ситуації, коли один потік захопив мютекс A і чекає на B, а другий захопив B і чекає на A.
- Існує ряд способів боротьби із загрозою такого deadlock (див., наприклад, Таненбаум, ”Сучасні операційні системи” (”Modern Operating Systems”)), наприклад єдине впорядкування блокувань, однак всі вони вразливі до помилок.
- Стандартна бібліотека надає два алгоритми, std::lock() та std::try_lock(), котрим передається група мютексів, і вони або захоплюють всі зразу, або не захоплюють жодного.
- Використання може виглядати якось так:

```
void  task(mutex& m1, mutex& m2){
      unique_lock<mutex> lck1 {m1,defer_lock_t};
      unique_lock<mutex> lck2 {m1,defer_lock_t};
      lock(lck1,lck2);
            .....................
}
```

- Або так:

```
void  task(mutex& m1, mutex& m2){
      lock  (m1, m2);
      unique_lock<mutex> lck1 {m1,adopt_lock_t};
      unique_lock<mutex> lck2 {m1,adopt_lock_t};
            .....................
}
```

- std::lock() працює, на свій розсуд викликаючи lock(), try_lock(), unlock() до переданих мютексів. Не повертає нічого.
- Якщо в процесі сталося виключення, він спочатку розблокує всі вже захоплені, потім повторно згенерує його.
- std::try_lock() пробує захопити мютекси у тому порядку, в якому вони йому передані.
- Якщо завершився успішно, повертає -1, інакше – номер мютекса, який захопити не вдалося (нумерація від нуля).
- Якщо захопити всі мютекси не вдалося, через те, що один із них вже заблокований або сталося виключення — звільняє всі захоплені.

std::scoped_lock (C++17)

- scoped_lock – узагальнення розглянутих раніше RAII обгорток, яка вміє захоплювати кілька м'ютексів зразу.
- ```
  template< class ... MutexTypes >
  class scoped_lock{
  public:
          explicit scoped_lock(MutexTypes&... Mtxes);
          scoped_lock( std :: adopt_lock_t , MutexTypes&... m );
          scoped_lock( const scoped_lock& ) = delete ;
          scoped_lock& operator=(const scoped_lock&) = delete ;

          ~scoped_lock() noexcept;
      .....
  };
  ```
- Для уникнення взаємоблокувань (deadlock) використовує алгоритм, аналогічний std::lock.
- Приклад:

  ```
  std :: scoped_lock lock(mux1, mux2, mux3);
  ```

  Зразу використано автоматичне виведення аргументів шаблону із C++17.
  ```

Екзотичні RAII-обгортки м’ютексів та інше

- ”Temporarily dropping a lock: The anti-lock pattern” – об’єкт, під час створення, віддає м’ютекс, а в деструкторі — захоплює його знову. Де такі об’єкти могли б бути корисними, див. далі – поведінку wait() умовних змінних.
- ”A pattern for obtaining a single value while holding a lock”.

Умовні змінні

- **Умовні змінні** (Condition Variables) використовуються для організації взаємодії між потоками. Також можуть служити для очікування протягом певних періодів.
- Один потік може заблокуватися на умовній змінній, до того поки не станеться якась подія в іншому потоці.
- Альтернативою було б активне очікування (busy loop) на якийсь прапорець, використання якого може бути багато менш ефективним, ніж використання умовних змінних.
- Оголошені відповідні класи у <condition_variable>.
- Використовують їх **разом** із мютексами!

- Виглядає використання наступним чином. Всі потоки повинні мати доступ і до умовної змінної і до мютекса:

```
// Global or equivalent scope
std :: mutex readyMutex;
std :: condition_variable readyCondVar;
```

- Потік, який чекає на подію, повинен зробити щось таке:

```
unique_lock<mutex> lck(readyMutex);
        .....................
readyCondVar.wait( lck );
```

- Потік, який хоче повідомити інші потоки про подію, повинен викликати `readyCondVar.notify_one();` або `readyCondVar.notify_all();`
- На жаль, **так просто воно не працюватиме (надійно)**.

Нюанси умовних змінних – spurious wakeup

- Найбільша проблема в тому, що потік може прокинутися марно — умова, насправді, не виконується (ще чи вже), але його збудили.
- ◇ Причина таких spurious wakeups пов'язана із особливостями платформ, уникнути їх важко, а наслідки від випадково пропущеної нотифікації були б важчими. Приклади причин – див. далі.
- Тому wait() **завжди повинна знаходитися в циклі**, умова якого перевіряє, чи це прокидання не є марним (spurious).
- Ще одна причина ситуації, коли потік прокинувся, а приводу для цього немає — хтось, поки потік спав, вже після сигналу, встиг опрацювати ситуацію, зробивши так, що умова прокидання знову не виконується.
- ◇ Наприклад, продуцент створив 5 порцій даних і викликав `readyCondVar.notify_all();`, збудивши 8 потоків. Тоді трьом із них нічого не дістанеться.
- Відповідно, код із попереднього прикладу мав би виглядати так:

```
std :: unique_lock<std :: mutex> ul(readyMutex);
while (! check_if_ready ())    readyCondVar.wait(ul );
```

де check_if_ready() дозволяє перевірити, чи справді все готово.

- Існує перевантаження wait(), яке приймає як аргумент предикат для перевірки – чи вже виконується умова:

```
std :: unique_lock<std :: mutex> ul(readyMutex);
readyCondVar.wait(ul , []{ return check_if_ready (); });
```

- Реалізація еквівалентна такій:

```
template< class Predicate >
void wait( std :: unique_lock<std :: mutex>& lock, Predicate pred ){
        while (! pred ()) {
            wait( lock );
        }
}
```

- C++20 додав wait() для умовних змінних std::condition_variable_any, який отримує stop_token, доданий разом із std::jthread. Детальніше див. розділ про std::jthread.

- Зауваження: разом із умовними змінними потрібно використовувати unique_lock — lock_guard недостатньо, так як відповідні функції (сімейство wait), повинні мати можливість маніпулювати мютексом.
- Справа в тому, що wait() звільняє мютекс, даючи можливість іншому потоку записувати у пам'ять даних, на які очікує цей. Перед поверненням wait() знову захоплює мютекс і код після виклику може продовжувати, знаючи, що має ексклюзивний доступ.
- Тому, зокрема, мютекси потрібно використовувати, навіть якщо змінна, якою маніпулюється, за допомогою умовних змінних, є атомарною!

Повноцінний приклад:

- Розглянемо повний приклад — задачу із N постачальників та M споживачів.
- Для простоти, нехай постачальники генерують випадкові числа із заданого інтервалу із випадковими затримками. Споживачі їх читають та виводять на консоль.
- Для спрощення коду, механізму завершення програми не передбачено – постачальники генерують обмежену кількість даних, а споживачі чекають в безмежному циклі.
- Підготовка:

```
std :: random_device rd;
std :: default_random_engine eng(rd ());

deque<int> queue;
mutex data_mutex;
 condition_variable   data_cond_var;
mutex io_mutex;

const int repeats = 30;
```

- Готуємо генератор випадкових чисел, чергу для даних та засоби організації доступу до неї — мютекс і умовну змінну, мютекс для серіалізації вводу-виводу.

Постачальник:

```
void provider (int delay_limit)
{
    std :: uniform_int_distribution <int> uniform_data_dist (1, 100);
    std :: uniform_int_distribution <int> uniform_delay_dist (1, delay_limit );
    for (int i=0; i<repeats ; ++i) {
        {
            std :: lock_guard<std :: mutex> lck(data_mutex);
            queue.push_back( uniform_data_dist (eng) );
        } // release data_mutex
        data_cond_var. notify_one ();
        std :: this_thread :: sleep_for (
                    std :: chrono :: milliseconds ( uniform_delay_dist (eng)) );
    }
}
```

- Тут достатньо lock_guard.
- Так як продукуємо за раз одну порцію даних, використано notify_one().

Споживач:

```
void consumer(int consumer_id){
    while (true) {
        int data;
        {
            std::unique_lock<std::mutex> lck(data_mutex);
            data_cond_var.wait(lck,[] { return !queue.empty(); });
            data = queue.front();
            queue.pop_front();
        } // release data_mutex
        {
            std::lock_guard<std::mutex> lck(io_mutex);
            std::cout << "Consumer " << consumer_id
                      << ": " << data << std::endl;
        } // release io_mutex
    }
}
```

- Кожен споживач виводить свій ідентифікатор та "пійманий" ним результат роботи постачальника.
- Для того, щоб вивід був акуратним, звертання до cout захищено io_mutex.

main()

```
int  n_provider_threads  = 10;
int  n_consumer_threads = 10;
vector<std :: thread>  provider_threads ;
vector<std :: thread> consumer_threads;

for(int  i  = 0;  i<n_consumer_threads; ++i){
      consumer_threads.emplace_back( consumer,  i   );
}

for(int  i  = 0;  i<n_provider_threads ;  ++i){
      provider_threads .emplace_back( provider ,  i*100 );
}

for(auto& thread  :  provider_threads )
      thread . join ();
for(auto& thread  :  consumer_threads)
      thread . join ();
```

- Споживачів створюємо першими, але, в принципі, якщо не боятися переповнення черги, це не принципово.

Приклад виводу:

```
Consumer 1: 43
Consumer 0: 16
Consumer 2: 33
Consumer 3: 84
Consumer 4: 7
Consumer 5: 98
Consumer 7: 80
Consumer 6: 34
Consumer 8: 34
Consumer 9: 65
Consumer 1: 65
Consumer 0: 87
Consumer 2: 7
Consumer 3: 64
Consumer 4: 7
Consumer 5: 58
Consumer 7: 74
Consumer 6: 92
Consumer 8: 57
```

Зверніть увагу на його недетерміністичність.

Подробиці умовних змінних

- std::condition_variable в ролі мютекса може використовувати лише `std::unique_lock<std::mutex>`.
- std::condition_variable_any – більш універсальний варіант, працюватиме із будь-яким засобом, що задовольняє вимогам BasicLockable — має методи lock() і unlock().
  - ◇ std::condition_variable часто ефективніші, але не завжди можна ними скористатися.
  - ◇ Семантика використання у них однакова.
- Умовні змінні — StandardLayoutType, їх заборонено копіювати чи переміщати.
- Конструювання — лише по замовчуванню.
- Знищувати безпечно лише якщо відбулася нотифікація всіх потоків, які очікують на цю змінну[33]
- Як і інші низькорівневі засоби, надає доступ до свого системного дескриптора:

```
native_handle_type   native_handle ();
```

[33] Це не означає, що вони негайно почнуть виконуватися — можуть очікувати на відповідний мютекс або диспетчер ще не розпочав їх виконання.

Надсилання повідомлень — нотифікація

- За нотифікацію відповідають дві функції:

  void notify_one ();
  void notify_all ();

- notify_one() — повідомляє один із потоків, що очікує на дану умовну змінну, notify_all() — повідомляє всі. Зрозуміло, що момент безпосереднього пробудження визначається багатьма іншими факторами, такими як заблоковані мютекси та поведінка диспетчера ОС.
- На момент виклику цих ф-цій відповідний мютекс не мав би бути заблокований — інакше потік, який очікує, буде розблоковано лише щоб він зразу заблокувався на спробі цей мютекс захопити[34]. (Див, зокрема, приклад вище).
- Виклики notify_one()/notify_all() і wait()/wait_for()/wait_until() глобально впорядковані — наприклад, notify_one() ніколи не буде затримано так, що вона збудить потік, який викликав wait() ПІСЛЯ виклику notify_one().

[34] Кажуть, POSIX навіть має спеціальну оптимізацію на такий випадок.

Очікування

- Функцій очікування є три: wait(), wait_for(), wait_until().

```
void wait( std::unique_lock<std::mutex>& lock );
template< class Predicate >
void wait( std::unique_lock<std::mutex>& lock, Predicate pred );

// В [] показано варіант із предикатом
template< class Rep, class Period [, class Predicate ] >
bool wait_for( std::unique_lock<std::mutex>& lock,
               const std::chrono::duration<Rep, Period>& rel_time
               [, Predicate pred ]);
template< class Clock, class Duration [, class Predicate ] >
bool wait_until ( std::unique_lock<std::mutex>& lock,
    const std::chrono::time_point<Clock, Duration>& timeout_time
    [, Predicate pred] );
```

- Всі вони є в двох варіантах — із предикатом і без. Функція із предикатом, після прокидання, використовує його, щоб перевірити, чи умова вже виконалася, і якщо ні – знову повертається до чекання. Тобто, для wait() еквівалентна:

```
while (!pred())
    wait(lock);
```

- **wait()** блокує поточний потік поки його не буде збуджено викликом notify_all() чи notify_one() (або, для варіанту без предикату — не станеться spurious wakeup).
- Розблоковує відповідний мютекс, захоплює його знову перед виходом.
- На моменти виклику мютекс має бути захопленим. Інакше — невизначена поведінка.
- Потік, який чекає і той, що нотифікує, повинні користуватися одним і тим же мютексом. Інакше — невизначена поведінка.
- **wait_for()** поводиться схоже, однак поточний потік також розблоковується коли минув переданий відрізок часу[35].
- Використовує монотонний (steady) годинник.
- Варіант без предикату повертає значення типу cv_status — no_timeout або timeout — вказуючи причину розблокування.
- Варіант із предикатом повертає true, якщо предикат на момент завершення повертав true, false — інакше.
- **wait_until()** схожа на wait_for(), але чекає до вказаного моменту і її годинник не обов'язково монотонний.
- Детальніше див. `http://en.cppreference.com/w/cpp/thread/condition_variable`

[35] Блокування може бути довшим, ніж вказаний інтервал — на розсуд диспетчера операційної системи.

std::notify_all_at_thread_exit()

- Для того, щоб автоматично повідомляти всі потоки, які очікують на даний по його завершенні, можна використовувати функцію `std::notify_all_at_thread_exit()`:

  void notify_all_at_thread_exit (std :: condition_variable & cond,
  std :: unique_lock<std :: mutex> lk);

  Нотифікація відбудеться після повного завершення потоку, зокрема — після знищення всіх Thread-local об'єктів.
- Мютекс має бути захопленим і має бути тим же, що використовується іншими потоками для очікування на нотифікацію від цього. (Інакше — undefined behavior.)
- Зазвичай ця функція — остання у detached-потоці.
- Детальніше див. `http://en.cppreference.com/w/cpp/thread/notify_all_at_thread_exit`.

Про марні прокидання – 1

- Цитуючи Раймонда Чена, `https://blogs.msdn.microsoft.com/oldnewthing/20180201-00/?p=97946`:
- "*One source of spurious wake-ups is the stolen wake-up, where a sleeping thread is woken, but by the time it gets a chance to run, another thread has already snuck in and taken the thing that the thread was waiting for, forcing the thread to go back and wait some more.*"
- "*Another source of spurious wake-ups is where there are a lot of threads waiting on the condition variable, and then there is a huge flurry of WakeConditionVariable calls. Normally, exactly one thread is woken for each call to WakeConditionVariable, but if there are a lot of wakes in rapid succession, the internal data structure doesn't have enough room in the "number of threads that need to be woken" bitfield to record the exact number, and the system says, "Well, I'll play it safe and just wake up everybody."*
  The "number of threads that need to be woken" is a bitfield rather than a full 32-bit value because all of the bookkeeping for a condition variable must fit inside a pointer-sized variable, so you have to be very frugal with how you use that limited space. And since condition variables explicitly permit spurious wake-ups, it's okay to be sloppy in keeping track of how many wake-ups are required, as long as you always err on the side of waking up too many people."

Про марні прокидання – 2

- І звідси `http://blog.vladimirprus.com/2005/07/spurious-wakeups.html`.
- ”*After asking on comp.programming.thread, I at least know the reason for Linux (thanks to Ben Hutchings). Internally, wait is implemented as a call to the ’futex’ system call. Each blocking system call on Linux returns abruptly when the process receives a signal – because calling signal handler from kernel call is tricky. What if the signal handler calls some other system function? And a new signal arrives? It’s easy to run out of kernel stack for a process. Exactly because each system call can be interrupted, when glibc calls any blocking function, like ’read’, it does it in a loop, and if ’read’ returns EINTR, calls ’read’ again.*
Can the same trick be used to conditions? No, because the moment we return from ’futex’ call, another thread can send us notification. And since we’re not waiting inside ’futex’, we’ll miss the notification. So, we need to return to the caller, and have it reevaluate the predicate. If another thread indeed set it to true, we’ll break out of the loop.”

Реалізація рекурсивного spinlock

```
class recursive_spinlock_t {
        std :: atomic_flag lock_flag ;
        int64_t recursive_counter = 0;
        std :: atomic<std :: thread :: id> owner_thread_id{std :: thread :: id {}};
public:
        recursive_spinlock_t () {
                lock_flag . clear ();
        }

        bool try_lock () {
                if (! lock_flag . test_and_set ( std :: memory_order_acquire)) {
                        owner_thread_id. store ( std :: this_thread :: get_id (),
                                                 std :: memory_order_release);
                } else {
                        if (owner_thread_id. load( std :: memory_order_acquire) !=
                                                 std :: this_thread :: get_id ())
                                return false ;
                }
                ++ recursive_counter ;
                return true;
        }
```

```
        void lock() {
                for (volatile size_t i = 0; !try_lock(); ++i)
                        if (i % 1000 == 0)
                                std::this_thread::yield();
        }

        void unlock() {
                assert(owner_thread_id.load(std::memory_order_acquire) ==
                                        std::this_thread::get_id());
                assert(recursive_counter > 0);

                if (--recursive_counter == 0) {
                        owner_thread_id.store(std::thread::id{},
                                        std::memory_order_release);
                        lock_flag.clear(std::memory_order_release);
                }
        }
};
```

- Взято, із змінами, тут: `https://habr.com/ru/post/328362/`.

Реалізація read-write mutex на базі std::mutex

- Розглянемо кілька реалізацій аналога std::shared_mutex.
- Написано за мотивами `https://eli.thegreenplace.net/2019/implementing-reader-writer-locks/`.
- Перша реалізація – найбільш прямолінійна.
- Зауважте, зробити лічильник нижче атомарним не допоможе. Слід захоплювати м'ютекс.

```
class simple_read_write_lock_t {
    std::mutex m;
    uint64_t reader_count = 0;
public:
    void lock_for_read(){
        std::lock_guard<std::mutex> lg{m};
        ++reader_count;
    }
    void unlock_for_read(){
        std::lock_guard<std::mutex> lg{m};
        --reader_count;
    }
    void lock_for_write(){
        while(true){
            m.lock();
            if(reader_count > 0){
                m.unlock();
            } else {
                break;
            }
        }
    }
    void unlock_for_write() {
        m.unlock();
    }
};
```

- Недолік попередньої реалізації – busy loop в lock_for_write(). Позбутися його можна за допомогою умовних змінних.

```
class adv_read_write_lock_t
{
    std :: mutex m;
    std :: condition_variable cv;
    uint64_t reader_count = 0;
public:
    void lock_for_read (){
        std :: lock_guard<std :: mutex> lg{m};
        ++reader_count;
    }
    void unlock_for_read (){
        std :: unique_lock<std :: mutex> lg{m};
        --reader_count;
        if (reader_count == 0){
            lg.unlock ();
            cv.notify_one ();
        }
    }
```

```
    void lock_for_write (){
        std :: unique_lock<std :: mutex> ul{m};
        while( reader_count > 0){
            cv.wait( ul );
        }
        ul . release (); // Інакше його розблокує. Але cv працює тільки із unique_lock,
                         // то мусимо хитрувати. Або use conditional_variable_any
    }

    void unlock_for_write () {
        m.unlock();
        cv. notify_one ();
    }
};
```

- Попередня реалізація надає перевагу читачам, може приводити до write starvation.
- Щоб цього позбутися – якщо з'явився запит на запис, нових читачів поки не пускаємо.
- Зауважте – мютекс тепер використовується лише для захисту стану класу, в блокуванні участі не бере.

```
class adv_read_write_lock_w_t{
    std::mutex m;
    std::condition_variable cv;
    uint64_t reader_count = 0;
    bool has_writer = false;
public:
    void lock_for_read(){
        std::unique_lock<std::mutex> ul{m};
        while(has_writer){
            cv.wait(ul);
        }
        ++reader_count;
    }
```

```
    void unlock_for_read(){
        std::unique_lock<std::mutex> lg{m};
        --reader_count;
        if(reader_count == 0){
            lg.unlock();
            cv.notify_all();
        }
    }
    void lock_for_write(){
        std::unique_lock<std::mutex> ul{m};
        while(has_writer){
            cv.wait(ul);
        }
        has_writer = true;
        while(reader_count > 0){
            cv.wait(ul);
        }
    }
    void unlock_for_write() {
        {
            std::lock_guard<std::mutex> lg{m};
            has_writer = false;
        }
        cv.notify_all();
    }
};
```

- Заради справедливості, хоча ретельних тестів продуктивності не проводилося, в базових тестах остання реалізація завжди сильно програвала.
- Крім того, всі три реалізації майже завжди (кілька винятків було) програвали як std::mutex, так і std::shared_mutex.

Бар'єр std::barrier<> (C++20)

- std::barrier<> – багаторазовий бар'єр.
- При створенні отримує величину лічильника та опціональну completion function.
- Коли вказана лічильником кількість потоків дійшла до бар'єра, виконується completion function, а бар'єр переходить у початковий стан (лічильник набуває початкового значення).
- Потоки (ті ж, що прибули в контрольну точку, або інші) можуть зачекати, поки не буде виконано completion function.
- Котрий із потоків виконуватиме її – не фіксується.
- Бар'єр – зручний спосіб узгодження роботи потоків[36].
- В std::experimental окремо виділяли barrier без completion function та std::experimental::flex_barrier із нею, C++20 їх об'єднав в один std::barrier<>.
- Станом на квітень 2020 не підтримується компіляторами.

[36] Найпростіший вироджений приклад – потоки чекають на запуск певної їх кількості, перш ніж продовжувати.

API

- Конструктор:

```
explicit barrier ( std :: ptrdiff_t num_threads );
```

- Точки синхронізації:
  - ◇ Прибути до точки синхронізації та зменшити лічильник на n:

```
[[ nodiscard ]] arrival_token arrive ( std :: ptrdiff_t n = 1 );
```

  - ◇ Зачекати, поки вказана початковим лічильником кількість потоків не пройде точку синхронізації:

```
void wait( arrival_token && arrival ) const;
```

  - ◇ Прибути до точки синхронізації (зменшити лічильник на 1) та зачекати, поки всі потоки її пройдуть:

```
void arrive_and_wait ();
```

  - ◇ Зменшити лічильник на 1 та зменшити початкове значення лічильника на 1 – після цього, щоб виконати completion function, потрібно буде, щоб прибуло на один потік менше.

```
void arrive_and_drop ();
```

  - ◇ Аналогічно, робить те, що описано в імені:

```
void arrive_and_wait ();
```

- Приклад із `https://www.justsoftwaresolutions.co.uk/files/cppcon_2019_concurrency_in_cpp20.pdf`:

```
int num_threads = ...;
void finish_task ();

std :: barrier <std :: function <void()>> b( num_threads, finish_task );

void worker_thread( std :: stop_token st , unsigned i ) {
        while( ! st . stop_requested () ){
                do_stuff ( i );
                b. arrive_and_wait ();
        }
}
```

std::latch (C++20)

- std::latch – ”одноразовий” варіант бар’єра.
- Станом на квітень 2020 не підтримується компіляторами.
- API:

```
constexpr explicit latch( std::ptrdiff_t expected );
void count_down( std::ptrdiff_t n = 1 );
bool try_wait() const noexcept; // true if counter == 0
std::latch::wait();
void arrive_and_wait( std::ptrdiff_t n = 1 );
```

Посилання стосовно бар'єрів із C++20

- Короткий огляд: `https://www.modernescpp.com/index.php/latches-and-barriers`.
- Документ-пропозиція в стандарт: `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0666r1.pdf`
- `https://en.cppreference.com/w/cpp/experimental/barrier`, `https://en.cppreference.com/w/cpp/experimental/latch`, `https://en.cppreference.com/w/cpp/experimental/flex_barrier`.
- `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1135r5.html` – "P1135R5: The C++20 Synchronization Library".

Семафори (C++20) DRAFT

- C++20 додав два види семафорів:

```
template<std::ptrdiff_t LeastMaxValue = /* implementation-defined */>
class counting_semaphore;
using binary_semaphore = std::counting_semaphore<1>;
```

- std::counting_semaphore – може мати довільне значення, аж по результат функції counting_semaphore::max().
- std::binary_semaphore – лише 0 або 1, але, на відміну від м'ютекса, захоплення та звільнення не прив'язані до конкретних потоків.
- Метод acquire() зменшує лічильник на 1, якщо він був більшим за 0, блокує інакше – до release().
- Метод release(n) збільшує лічильник на n. По замовчуванню – на 1.
- Є також методи try_acquire(), try_acquire_for(), try_acquire_until().
- LeastMaxValue задає мінімальне максимальне значення лічильника – counting_semaphore::max() може повернути його або більше значення. Однак, за перевищенням counting_semaphore::max() повинен слідкувати програміст.
- Станом на квітень 2020 не підтримується компіляторами.

Посилання стосовно семафорів із C++20

- Документ-пропозиція в стандарт: `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0514r4.pdf` (застаріла)
- `https://en.cppreference.com/w/cpp/thread/counting_semaphore`,
- `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1135r5.html` – ”P1135R5: The C++20 Synchronization Library”.

atomic_wait, atomic_notify_XXX і компанія (C++20) DRAFT

- В C++20 з'явилася можливість засинати та прокидатися на атомарних змінних (всі – шаблони із одним аргументом T):

```
void atomic_wait(const volatile atomic<T>* object,
                 typename atomic<T>::value_type old);
void atomic_wait(const atomic<T>* object,
                 typename atomic<T>::value_type old);
void atomic_wait_explicit (const volatile atomic<T>* object,
                           typename atomic<T>::value_type old,
                           memory_order order);
void atomic_wait_explicit (const atomic<T>* object,
                           typename atomic<T>::value_type,
                           memory_order order );
void atomic_notify_one( volatile atomic<T>*);
void atomic_notify_one(atomic<T>*);
void atomic_notify_all ( volatile atomic<T>*);
void atomic_notify_all (atomic<T>*);
```

- Аналогічні методи додано до std::atomic<> – wait(), notify_one(), notify_all().

- Можливість засинати на атомарних змінних дозволяють більш ефективно відстежувати їх зміни, ніж це можна зробити явним опитуванням.
- explicit-варіанти задають впорядкування пам'яті явно.
- wait() порівнює атомарну змінну із old, побітово.
  - Якщо вони відрізняються – повертає керування.
  - Якщо ні – засинає до notify або spurious wakeup, і перевіряє знову, в циклі.
- Тобто, завершується ця функція, лише якщо значення змінилося – незалежно від марних прокидань будь-якої природи.
- Через ABA проблему, можуть пропускати переходи A-B-A.
- `std::atomic_signed_lock_free` та `std::atomic_unsigned_lock_free` – атомарні типи, найбільш ефективні для таких операцій.

Посилання про atomic_wait, atomic_notify_XXX і тощо з C++20

- `https://en.cppreference.com/w/cpp/atomic/atomic`
- `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1135r5.html` – ”P1135R5: The C++20 Synchronization Library”.

- Серія статей про створення потокобезпечних структур даних:
- `https://habrahabr.ru/post/328348/` – ”Делаем любой объект потокобезопасным.”
- `https://habrahabr.ru/post/328362/` – ”Ускоряем std::shared_mutex в 10 раз”. Зокрема, дуже хороший опис впорядкувань звертання до пам’яті, із гарними ілюстраціями.
- `https://habrahabr.ru/post/328374/` – ”Потокобезопасный std::map с производительностью lock-free map”
- ”Optimizing a ring buffer for throughput” – аналіз простої lock-free структури даних, кільцевого буферу з одним постачальником і одним споживачем.

Hazard pointers – C++26

- В C++26 запропоновано додати підтримку hazard pointers.
- Це інструмент керування динамічної пам'яттю для lock free структур даних. Зокрема, вони допомагають вирішити:
  - ABA-проблему.
  - проблему видалення вузлів структури, які використовуються іншими потоками.
- Для цього, за використання підходу hazard pointers, кожен потік підтримує список вказівників на вузли, з якими він працює[37].
  - Інші потоки не видаляють та не модифікують елементи із цього списку.
  - Вони також не можуть модифікувати цей список, але можуть його читати.
- Спеціалізований потік, або кожен з потоків, підтримує список звільнених вузлів, і видаляє їх, коли жоден потік не містить його в своїх hazard pointer-списках.
- Документ з пропозицією: "Hazard Pointers for C++26". Моделювалися за hazard pointers з Folly.
- Короткий огляд є тут, див. також Wiki.

[37] Розмір цього списку може бути обмеженим одним або двома елементами.

- Приклад з пропозиції:

```
struct Data : hazard_pointer_obj_base<Data>
{ ..... };

atomic<Data*> pdata_;

// Читання може працювати паралельно
template <typename U, typename Func>
U reader_op(Func userFn) {
    hazard_pointer h = make_hazard_pointer();
    Data* p = h.protect(pdata_); // Звертатися за допомогою p -- безпечно.
    // Функція з тривалою операцією:
    return userFn(p);
    // Не блокуватиме запис в саму структуру даних та видалення.
} // Завершення захищеного періоду за допомогою RAII


// Можна викликати паралельно з читання.
void writer(Data* newdata) {
        Data* old = pdata_.exchange(newdata);
        old->retire(); // Почистити не використовувані обєкти'
}
```

- Призводить до появи кількох "епох" структури даних – до можливості наявності кількох версій структури даних в пам'яті.

Read-Copy-Update (RCU) – C++26

- В C++26 також запропоновано додати підтримку Read-Copy-Update, які hazard pointers, змодельовану за бібліотекою Folly.
- RCU – підхід синхронізації для структур, що є реалізовані через зв'язні списки. Це можуть бути черги, дерева, хеш-таблиці.
- Коли відбувається додавання або видалення елементу в таку структур, гарантується, що всі потоки бачитимуть або старий варіант, або новий – але не якийсь проміжний стан.
- Код, що працює з даними такої структури, робить їх копію, модифікує і додає назад в структуру – звідти й назва.
- Детальніше див. цю серію статей: "What is RCU, Fundamentally?", "What is RCU? Part 2: Usage", "RCU part 3: the RCU API".
- Документ-пропозиція: "Read-Copy Update (RCU)", який, поміж іншого, посилається, наприклад, на `https://liburcu.org`.
- Реалізація з Folly та відповідна документація.

- Приклад з пропозиції:

```
struct Data : std::rcu_obj_base<Data>
{ ..... };

template <typename Func>
Result reader_op(Func fn) {
        std::scoped_lock l(std::rcu_default_domain());
        Data* p = data_;
        // Функція не повинна надовго блокувати, чи викликати
        // rcu_synchronize(), rcu_barrier(), rcu_retire()
        return fn(p);
} // RAIIвиклик− rcu_synchronize()

void update(Data* newdata) {
        Data* olddata = data_.exchange(newdata);
        olddata->retire();
}
```

- std::rcu_default_domain() повертає статичний об'єкт rcu_domain, який визначає регіон захисту RCU. В Folly можна створювати й інші регіони, в стандарті поки ні.
- При цьому, std::rcu_synchronize() – дочекатися кінця покоління, іншими словами – щоб всі читачі завершили, std::rcu_barrier() – дочекатися видалення застарілих об'єктів.
- Можливий також не-інтрузивний варіант – без успадкування від std::rcu_obj_base.

Атомарні fetch_max та fetch_min

- Функціх fetch_max() та fetch_min() атомарно обирають, відповідно, більший чи менший, із вмісту атомарної змінної чи аргументу і зберігає його назад.

  template<**class** T>
  T atomic_fetch_max(atomic<T>∗, **typename** atomic<T>::value_type) noexcept;

  template<**class** T>
  T atomic_fetch_max_explicit(atomic<T>∗, **typename** atomic<T>::value_type, memory_order) noexcept;

  // Те ж для *min*.

- ◇ Турбується про деякі нетривіальні подробиці.
- Детальніше див. ”Atomic minimum/maximum”.
- Не підтримують floating point. Щодо них див. ”Atomic floating-point min/max{}”.

std::execution executors – C++26?

- Є певна надія, що в C++26 потраплять екзекутори. Можливо – якась частина...
- Документ з пропозицією: std::execution.
- Приклад звідти:

```
using namespace std::execution;

scheduler auto sch = thread_pool.scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(add_42).value();
```

- Див. також на cppreference.

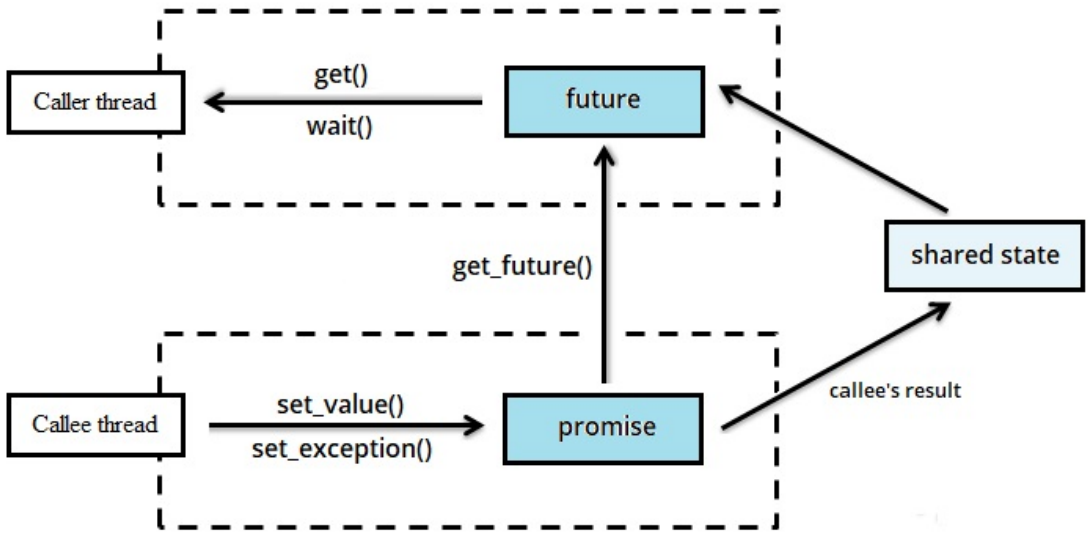
- std::execution – оригінальний документ, дуже громіздкий
- New C++ Sender Library Enables Portable Asynchrony – експериментальна реалізація від Nvidia. Також, код.
- libunifex – реалізація від Фейсбук.
- ”A Universal Async Abstraction for C++”
- ◇ A Universal I/O Abstraction for C++ – приклад використання з io_uring.
- ”Trying out C++26 executors” – ілюстрація, що підхід стандарту, все ж, громіздкий[38].
-

[38] Автор презентації все задумується – за тим якась глибина, чи це просто (дефензивний) дизайн комітетом?

C++29

- Запропоновано ”P3125R6: constexpr pointer tagging” – портабельну підтримку збереження інформації в молодших бітах вказівників.

- Одна із основних проблем багатопоточності — необхідність спільного використання пам'яті. C++ додає ще одну — потребу "прокидати" виключення між потоками.
- Для вирішення цих проблем вводиться поняття спільного стану (shared state). Для доступу до нього надається дві сутності: future дозволяє отримувати (асинхронно) значення чи виключення, збережене в спільному стані, а promise — встановити їх.
- При цьому, для обміну даних, користуючись парою future-promise явні блокування не потрібні.



Зображення звідси: `https://www.codeproject.com/Articles/1278737/Programming-Concurrency-in-Cplusplus-Part-2`.

- Взагалі кажучи, спільним станом можна користуватися і в одному потоці, хоч особливого сенсу в тому немає.
- І те і те є шаблоном, параметр якого вказує, аргумент якого типу передаватиметься.
- promise<> має методи set_value() і set_exception(), котрі дозволяють встановити значення спільного стану.
- У свою чергу, future<> має метод get(), котрий отримує передане значення.
- Якщо значення ще не готове — відповідний promise<> не виконав жодного set, get() блокує відповідний потік, поки дані не будуть готові.

```
future<int> number_fut;
future< vector<double> > vector_fut ;
```

Приклад – головна програма

```
try{
    promise<int> result_promise;
    std::thread thread1(worker, ref(result_promise)); //! Notice ref!
    thread1.detach(); // Щоб не треба було join() викликати

    future<int> result(result_promise.get_future());

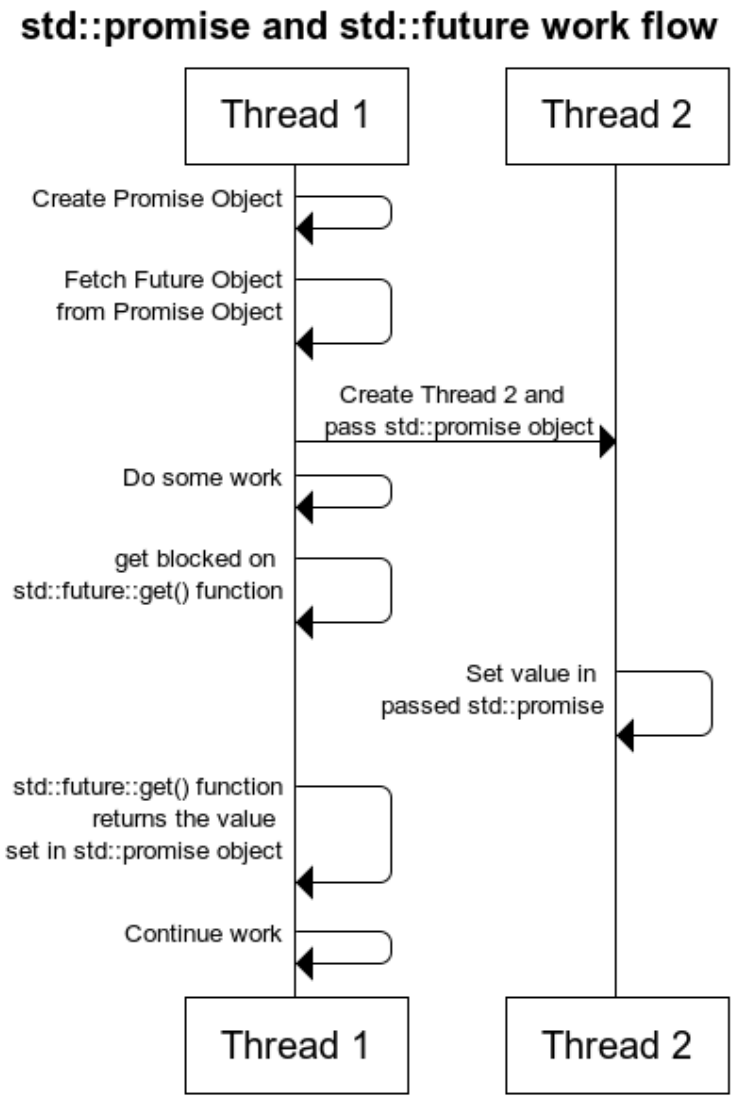
    cout << "Found: " << result.get() << endl;
}
catch(std::exception &e){
    cout << "Exception: " << e.what() << endl;
}
```

- Потоку передається посилання на об'єкт-обіцянку — promise.
- Потім створюється відповідний future, пов'язаний із ним.
- Виклик його методу get() поверне результат або кине виключення, що виникло в потоці.
- За потреби цей метод дочекається наявності результату – є синхронним.
- **Увага** — використовувати detach(), на загал, погана практика. Тут лише для простоти.

Функція, що виконується в потоці: worker

```
void worker(promise<int>& result)
{
    try{
        int x;
        cout << "Enter even number: ";
        cin >> x;
        if (x%2 == 0)
        {
            result.set_value(x);
        }else
        {
            throw runtime_error("Odd number!");
        }
    }
    catch (...)
    {
        result.set_exception( std::current_exception() );
    }
}
```

Підсумовуючи



Зображення взято тут: `https://thispointer.com/c11-multithreading-part-8-stdfuture-stdpromise-and-returning-values-`

- Насправді, в попередньому прикладі, detach() було викликане, щоб спростити код — копіювати thread не можна, возитися із вказівниками — неохота, а щоб викликати join(), треба щоб об'єкт thread був доступний і після нього.
- Вирішити цю проблему можна, згадавши, що потоки можна переміщати:

```
std :: thread   thread1 ;
try{
    promise<int>  result_promise ;

    thread1  =  std :: thread  (  worker,  ref( result_promise )  );  // [1]

    future<int>  result (  result_promise . get_future ()  );
    cout  << "Found: "  <<  result . get ()  << endl;
}
catch( std :: exception  &e)
 {
    cout  << "Exception:  "  << e.what()  << endl;
}
thread1 . join ();  // [2]
```

- В рядку [1] щойно створений тимчасовий об'єкт std::thread буде переміщено в thread1. Це дозволить дочекатися потоку в рядку [2].

- Переміщати є сенс і самі обіцянки-promise:

```
void worker(promise<int> result ) { ..... }

                ..........................................

std :: thread thread1 ;
try{
    promise<int> result_promise ;
    future<int> result ( result_promise . get_future () );

    thread1 = std :: thread ( worker, move(result_promise) ); // [1]

    cout << "Found: " << result . get () << endl;
}
catch( std :: exception &e)
 {
    cout << "Exception: " << e.what() << endl;
}
thread1 . join ();
```

- worker тепер приймає promise за значенням, а move() у рядку [1] наказує виконати переміщення.

std::promise

- Клас std::promise надає можливість зберігати дані у shared state.
- Існує у двох спеціалізаціях:

```
template< class  R > class  promise;
template< class  R > class  promise<R&>;
template<>           class  promise<void>;
```

  Перша дозволяє обмінюватися даними, а друга — повідомляти про події без стану. Виключення можуть передаватися як однією, так і іншою.
- promise може виконувати три операції над спільним станом:
  - Підготувати (make ready) — зберегти результат або виключення в спільний стан.
  - Звільнити (release) — відмовитися від доступу до спільного стану. Якщо це було останнє посилання на нього, спільний стан знищується. Крім випадку спільного стану, створеного незавершеним std::async, не блокує.
  - Покинути спільний стан (abandon). Зберігає в нього виключення `std::future_error` з кодом `std::future_errc::broken_promise`, підготувавши до використання, звільняє.
- Схожі інструменти є в багатьох мовах програмування. Детальніше див. `https://en.wikipedia.org/wiki/Futures_and_promises`.

std::promise — створення, копіювання, переміщення, знищення

- Існують конструктори по замовчуванню, із можливістю вказати алокатор та конструктор переміщення.
- Конструктор копіювання та оператор присвоєння відсутні — deleted.
- Однак, існує оператор переміщення та метод swap().
- Якщо об’єкт не був готовим, деструктор виконує abandon — покидає стан, фактично передаючи виключення `std::future_error` з кодом `std::future_errc::broken_promise`. Якщо готовий — звільняє спільний стан (release).

std::promise — get_future()

- Метод **get_future()** — отримати future, що відповідає тому ж спільному стану, що this.

```
std :: future<T> get_future ();
```

- Викликати можна лише раз.
- Якщо promise відповідає об'єкту без спільного стану або за повторного виклику — кидає виключення std::future_error, із кодом помилки std::future_errc::no_state в першому випадку та std::future_errc::future_already_retrieved в другому.

std::promise — збереження результату

- Метод set_value() підготовлює спільний стан:

```
void set_value( const R& value ); // Член лише загального шаблону
void set_value( R&& value ); // Член лише загального шаблону
void set_value( R& value ); // Член лише спеціалізації promise<R&>
void set_value (); // Член лише спеціалізації promise<void>
```

- Кидає виключення[39], якщо немає спільного стану або значення вже записано — встановити значення спільного стану можна лише раз!
- `promise<void>` — важливий частковий випадок, дозволяє передавати сигнали між потоками. Ілюстрацію до використання див. частину ”Using promise<void> to signal” прикладу `mt_threads11_future_promise_demo`.
- Метод `set_value_at_thread_exit()` існує в тих же чотирьох варіантах, що й set_value(), також зберігає значення, але спільний стан не стає готовим — його готовність відкладається до моменту завершення потоку. Стан буде підготовлено після знищення всіх TLS змінних.
- Ці операції встановлення ніколи не створюватимуть гонитви даних із відповідними операцією читання із future.

[39] std::future_error, із кодом no_state або promise_already_satisfied. Також, пропустить виключення конструкторів копіювання чи переміщення даних

std::promise — передача виключень

- Метод, який дозволяє передати виключення за межі потоку:

  void set_exception (std :: exception_ptr p);

- `std::exception_ptr` — спеціальний розумний вказівник на виключення. Дозволяє передавати його між функціями та потоками. Детальніше див. `http://en.cppreference.com/w/cpp/error/exception_ptr`
- Кидає виключення, якщо спільного стану (куди можна зберегти виключення) немає, або стан вже було підготовлено.
- Приклад використання:

```
void worker(promise<int> result ){
    .....
  catch (...){
      result . set_exception ( std :: current_exception () );
  }
```

- Відтерміноване до знищення TLS-змінних потоку встановлення:

  void set_exception_at_thread_exit (std :: exception_ptr p)

- Ці операції ніколи не створюватимуть гонитви даних.

std::future

- Кажучи загально, клас future надає доступ до результатів асинхронних операцій. Зокрема, може створюватися вже розглянутим std::promise, а також std::async, std::packaged_task.
- Як і promise, існує в трьох варіантах:

```
template< class  T > class   future ;
template< class  T > class   future <T&>;
template<>           class   future <void>;
```

- Якщо результат у відповідному спільному стані ще не готовий, може блокувати до готовності.
- Об'єкт std::future має виключний доступ на читання до пов'язаного із ним спільного стану (shared state). (Див. також std::shared_future.)
- Існує конструктор по замовчуванню, який не пов'язаний із жодним станом та конструктор переміщення.
- Існує також оператор присвоєння із переміщенням.
- Конструктор копіювання та оператор присвоєння заборонені.
- Деструктор не блокується, крім випадку, коли спільний стан було створено викликом async(), результат ще не готовий і це останнє посилання на спільний стан. (Уточнено в C++14).

std::future — операції із спільним станом — 1

- Метод valid() дозволяє перевірити, чи future посилається на спільний стан:

  bool valid () **const**;

  Не-валідний об'єкт можна знищувати, перевіряти валідність та переміщати у нього, але на тому все – решта операцій є undefined behavior. Реалізація може (однак, не зобов'язана) кидати при цьому виключення std::future_error із кодом no_state.

- Найважливіший метод get():

  T get (); // Лише в загальному шаблоні *future*<>, переміщає у результат
  T& get (); // Лише в спеціалізації *future*<*T&*>
  void get (); // Лише в спеціалізації *future*<*void*>

  Дозволяє отримати результат. За потреби – очікує на готовність, так, ніби було викликано wait().

- Викликати можна **лише раз!** Після виклику valid() повертатиму false.

- Якщо в спільному стані було збережено виключення, get() його кине в поточному потоці.

std::future — операції із спільним станом — 2

- Метод share() дозволяє передати спільний стан під керування shared_future<T>:

```
std :: shared_future <T> share ();
```

- Метод wait() очікує, блокуючи, поки результат не буде готовим:

```
void wait () const;
```

- Наступні методи дозволяють очікувати лише вказаний час:

```
template< class Rep, class Period >
std :: future_status wait_for ( const std :: chrono :: duration <Rep,Period>& t
template< class Clock, class Duration >
std :: future_status wait_until ( const std :: chrono :: time_point <Clock,Dura
```

  Очікують поки результат не буде готовим, але не більше вказаного часу або не далі вказаного моменту. Результат вказує причину продовження роботи:
  - future_status::deferred — обчислення ще не розпочато.
  - future_status::ready — дані готові.
  - future_status::timeout — стався таймаут, дані на момент завершення — не готові.
- Традиційно, годинник wait_for() — монотонний, а wait_until() — не обов'язково. Через особливості диспетчеразації чи недостачу ресурсів, обидві функції можуть очікувати довше, ніж вказано.

std::shared_future

- На відміну від future, який для даного стану може існувати лише в одному екземплярі (який можна переміщати), std::shared_future дозволяє кільком об'єктам (із різних потоків) працювати із тим же спільним станом — очікувати на результат та отримувати його.
- Доступ із різних потоків безпечний, якщо кожен потік використовує свою копію об'єкту.
- Може конструюватися із future з переміщенням, чи з іншого shared_future, копіюючи чи переміщаючи:

```
shared_future ();
shared_future ( const shared_future & other );
shared_future ( std :: future<T>&& other );
shared_future ( shared_future && other );
```

- Існують оператори присвоєння та присвоєння із переміщенням.
- Деструктор не робить нічого, якщо на спільний стан посилаються ще якісь об'єкти, знищує його, якщо ні.
- Може використовуватися для нотифікації багатьох потоків, по аналогії із `std::condition_variable::notify_all()`.

std::shared_future — використання

- Методи valid(), wait(), wait_for(), wait_until() поводяться аналогічно до відповідних методів std::future.
- Їм можна безпечно викликати із різних потоків для різних об’єктів, пов’язаних із одним спільним станом, **однак**, виклик із різних потоків для одного об’єкту не є безпечним!
- Метод get() теж, на загал, схожий на аналогічний із std::future, однак, його можна викликати декілька раз.

std::future_error і std::future_errc

- Для обробки помилок асинхронних засобів C++11 та спільних станів, використовується виключення std::future_error, яке є нащадком std::logic_error.
- Формально, його метод code() повертає платформонезаленжий код помилки std::error_code, однак, фактично результат буде належати до сумісного із ним по кодах std::future_errc :

```
enum class future_errc {
    broken_promise              = /* implementation-defined */,
    future_already_retrieved    = /* implementation-defined */,
    promise_already_satisfied   = /* implementation-defined */,
    no_state                    = /* implementation-defined */
};
```

- C++14 виправив один недогляд у оголошенні цього перерахування — всі коди ненульові (для std::error_code/std::error_condition нульовий код означає відсутність помилки).

Concurrency TS

- В поточному стані std::future доволі примітивні.
- Concurrency TS пропонує ряд розширень, які поки не ввійшли в жоден стандарт.
- По перше, це так звані continuations – можливість сказати, що робити, коли future буде задоволено (fulfiled).
- По друге, when_all() та when_any() – функції, що повертають future, який буде задоволено, коли всі або довільний із переданих буде готовим.
- Станом на березень 2020 підтримка їх компіляторами практично відсутня, тому тут – короткий огляд, а застосування цих ідей буде проілюстровано на прикалі Facebook folly та boost::thread.

Continuations

- Continuation (продовження) в програмуванні – можливість зберегти стан програми, щоб продовжити виконання із цього ж місця потім. Детальніше див. `https://en.wikipedia.org/wiki/Continuation`.
- В мовах, що підтримують future або аналогічні засоби, continuation – можливість виконати якусь дію, коли future було виконано.
- Дозволяє більш зручно комбінувати програму із готових блоків.
- Спрощує проблему callback hell для асинхронного програмування.
- На прикладі Concurrency TS:

```
using exp = std::experimental;
exp::future<double> get_result();
exp::future<int> analise_result( exp::future<double> );

auto intermediate_res = get_result();
auto final_res = intermediate_res.then( analise_result );
```

Особливості C++ Concurrency TS continuations

- Додавати до std::experimental::future можна лише одне продовження, але так як then() теж повертає future, до нього можна додати ще одне і т.д.
- Вихідне std::experimental::future після цього перестає бути валідним – valid() поверне false.
- Вихідне std::experimental::shared_future залишається валідним.
- До std::experimental::shared_future можна додавати кілька продовжень.
- Потік, у якому виконуватиметься continuation – на розсуд runtime.
- Якщо функція-продовження повертає тип U, і він не є std::future<T> для якогось типу T, результатом then() буде std::future<U>, інакше – власне, std::future<T> – сам U. Тобто, future схлопуються в одне.
  - Це так зване неявне розгортання (implicit unwrapping).
  - Те ж буде для std::experimental::shared_future.

Проблеми

- Через відсутність в стандарті екзекуторів, є проблема контролю – який саме потік виконуватиме continuation.
- Тому із TS в стандарт вони поки не перекочують.
- Езекутори заплановані в 23-й стандарт. На крайній випадок – в 26-й.
- Експериментальна реалізація пропозицій: `https://github.com/facebookexperimental/libunifex`.
- Ілюстрація відповідних ідей буде на інших бібліотеках – facebook Folly[40], asynqro[41], Async++[42] тощо.
- Критика std:future із C++11: `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0701r1.html`.
  - Короткий переказ: `https://www.modernescpp.com/index.php/the-end-of-the-detour-unified-futures`.
- Критична стаття, із нижчим статусом, але з пропозицією альтернативної реалізації: `https://blog.thousandeyes.com/futures-library-asynchronous-programing-cplusplus/`.
- `https://vittorioromeo.info/index/blog/zeroalloc_continuations_p0.html` і продовження за посиланнями там.

[40] `https://github.com/facebook/folly`, `https://github.com/facebook/folly/blob/master/folly/docs/Futures.md`.

[41] `https://github.com/dkormalev/asynqro`

[42] `https://github.com/Amanieu/asyncplusplus`

Готові future

- `std::make_ready_future` дозволяє створити future зразу вдоволеним.
- ◇ Приклад[43]:

```
future<int> compute(int x) {
    if (x < 0) return make_ready_future<int>(-1);
    if (x == 0) return make_ready_future<int>(0);
    future<int> f1 = async([]() { return do_work(x); });
    return f1;
}
```

- Аналогічно, `exceptional_future` дозволяє зробити future із виключенням:

```
return make_exceptional_future<T>(std::current_exception());
return make_exceptional_future<T>(
                                    std::runtime_error("Smth bad is
                                );
```

[43] Взято тут: `https://www.modernescpp.com/index.php/std-future-extensions`.

std::experimental::when_all

- when_all повертає future, яке стане вдоволеним, коли буде вдоволено всі future із переданого йому списку.
- Concurrency TS пропонує два перевантаження:

```
template < class  InputIt  >
auto when_all( InputIt  first ,  InputIt  last )
    -> future<std :: vector<typename std::  iterator_traits  <InputIt >:: value

template < class ...  Futures  >
auto when_all(Futures&&...  futures )
    -> future<std :: tuple<std :: decay_t<Futures >...>>;
```

- Перший варіант приймає інтервал і повертає vector future<T>, у який було переміщено вихідні future<T>, якщо інтервал містив future<T>, або vector копій shared_future<T>, якщо на вході були вони.
- Другий – довільний список future та shared_future для довільних типів, результат повертається кортежем.
- На жаль, типи вони повертають довжелезні, щось типу:

```
exp :: future< std :: tuple  < exp :: future<T1>, exp :: future<T2>,
                exp :: shared_future<T3>, exp :: future<T4>>
                >
```

Приклад

- Приклад не випробовувався через відсутність підтримки компіляторами:

```
auto res1 = async( get_res_1 ,     .....     args     .....);
auto res2 = async( get_res_2 ,     .....     args     .....);
auto res3 = async( get_res_2 ,     .....     args     .....);
auto  results  = exp :: when_all(  std :: move(res1 ),  std :: move(res2 ),
                                   std :: move(res3)  );
 results  . then( merge_results );
```

std::experimental::when_any

- when_any повертає future, яке буде вдоволеним, коли задовольниться хоча б одне із переданих їй future/shared_future.
- Запропоноване оголошення:

```
template < class Sequence >
struct when_any_result {
    std :: size_t index;
    Sequence futures ;
};
template < class InputIt >
auto when_any(InputIt first , InputIt last )
  -> future<when_any_result<std:: vector<typename
          std :: iterator_traits <InputIt >:: value_type>>>;

template < class ... Futures >
auto when_any(Futures&&... futures )
  -> future<when_any_result<std:: tuple<std :: decay_t<Futures >...>>>;
```

- Поведінка схожа на when_any, зокрема, в when_any_result::futures переміщаються/копіюються вхідні future/shared_future.
- when_any_result::index містить індекс вдоволеного future.

Приклад

- Приклад[44]:

```
vector<future<int>> v{  ....  };
auto future_any = when_any(v.begin(), v.end());

when_any_result<vector<future<int>>> result = future_any.get();

future<int>& ready_future = result.futures[result.index];

auto myResult= ready_future.get();
```

[44] Взято тут: https://www.modernescpp.com/index.php/std-future-extensions.

Сторонні бібліотеки

- Окрім facebook::folly та asynqro, див. також QFutureWatcher із Qt: `https://doc.qt.io/qt-5/qfuturewatcher.html`.
- Цей клас інтегрований в систему сигналів-слотів Qt, покладається на неї. У використанні часто зручніший (ціною накладних витрат на обробку сигналів), ніж засоби типу описаних вище.
- Приклад із офіційної документації:

```
// Instantiate the objects and connect to the finished signal.
MyClass myObject;
QFutureWatcher<int> watcher;
connect(&watcher, SIGNAL(finished()),
                  &myObject, SLOT(handleFinished()));

// Start the computation.
QFuture<int> future = QtConcurrent::run(...);
watcher.setFuture( future );
```

std::packaged_task

- Вручну керувати створенням і взаємодією задачі-обіцянки-результату (task/thread-promise-future) часто незручно.
- Для спрощення типового їх використання стандартна бібліотека надає клас packaged_task.
- Конструктору цього об'єкту може приймати Callable, після чого можна отримати future() що відповідає спільному стану.
- Для запуску задачі слід викликати цей об'єкт як функцію, із аргументами задачі.
- Результат роботи цієї функції або кинуте нею виключення буде збережено у відповідний спільний стан.
- Зокрема, зробити це можна в окремому потоці.
- **Увага!** Сама по собі, "запакована задача" виконуватиметься в тому потоці, де її запустили.
- На загал, packaged_task — технічний інструмент для реалізації більш високорівневих засобів, типу std::async(). Являє собою контейнер для Callable, схожий на std::function.

Приклад packaged_task

```
unsigned long worker(unsigned long from, unsigned long to){
    return printPrime(from, to); // Some function
}

        std :: packaged_task<unsigned long(unsigned long,unsigned long)>
                                        task(worker);

        future<unsigned long> result { task.get_future() };

#ifdef USE_TASK_DIRECTLY
        task(1, max_number);    //! Працюємо в цьому потоці
#else
        //! Або запускаємо інший потік для задачі
        std :: thread task_tread(std :: move(task), 1, max_number);
        task_tread.join();
#endif
        cout << "Found: " << result.get() << endl;
```

- Якщо задачу запустити як "task(1, max_number);", вона зразу почне виконуватися, але її можна "згодувати" іншому потоку — для досягнення паралельної обробки.

std::packaged_task — створення та знищення

- Прототипи конструкторів змінювалися і в C++14 і в C++17. Тому не наводитиму їх — детальніше див. `http://en.cppreference.com/w/cpp/thread/packaged_task/packaged_task`. Існує три основних конструктори — конструктор по замовчуванню, який створює packaged_task без пов'язаної задачі чи спільного стану, конструктор, що приймає Callable, та конструктор, що приймає аллокатор спільного стану і Callable.
- Конструктор копіювання та оператор присвоєння заборонені.
- Існує конструктор переміщення та оператор присвоєння із переміщенням.
- Деструктор покидає спільний стан та знищує збережену задачу. Якщо спільний стан не було підготовлено, то в ньому буде збережено виключення std::future_error із кодом помилки std::future_errc::broken_promise.

Робота із std::packaged_task — 1

- Метод get_future() повертає future, пов'язане із спільним станом, у який зберігатиметься результат роботи задачі або кинуте нею виключення.
- **Увага!** Викликати get_future() можна лише раз для кожної packaged_task. При спробі кине виключення std::future_error із кодом future_already_retrieved.
- Спроба викликати get_future() за відсутності спільного стану, кине це ж виключення, із кодом no_state.
- Оператор виклику функції дозволяє запустити задачу. Результат чи виключення зберігаються у спільному стані. Ніякої явної синхронізації для доступу до результатів за допомогою отриманого future не потрібно.
- Метод `make_ready_at_thread_exit()` не підготовлює спільний стан поки не завершиться потік, де виконується задача (і не буде знищено TLS змінні). Детальніше див. приклад тут `http://en.cppreference.com/w/cpp/thread/packaged_task/make_ready_at_thread_exit`.

Робота із std::packaged_task — 2

- Метод valid() повертає true, якщо із packaged_task пов’язано спільний стан, false інакше.
- Метод swap() та однойменна функція обмінюють дві packaged_task між собою.
- Метод reset() — перевантажує стан, покидаючи результат попереднього виконання. Спільний стан конструюється заново.

std::async()

- Розглянуті досі засоби доволі низькорівневі. Поверх них стандартна бібліотека надає високорівневий інструмент — функцію async(), котра дозволяє асинхронно запустити задачу та отримати її результат, не вникаючи в подробиці асинхронного запуску.
- Виглядати це може якось так:

```
unsigned long worker(unsigned long from, unsigned long to){
    return printPrime(from, to); // Some function
}

.....
future<unsigned long> result { async( worker, 1, max_number ) };
cout << "Found: " << result.get() << endl;
```

- async() передає ("форвардить") свої аргументи задачі і повертає future для доступу до результату.
- Момент запуску залишається на розсуд реалізації. Деякі подробиці – нижче.

Запуск async()

- Прототип async()[45]:

```
template< class Function, class ... Args>
std :: future <typename std:: result_of <Function(Args ...)>:: type>
      async( Function&& f, Args&&... args );

template< class Function, class ... Args >
std :: future <typename std:: result_of <Function(Args ...)>:: type>
      async( std :: launch policy , Function&& f, Args&&... args );
```

- Повертає future<>, який можна використати для доступу до аргументів.
- Перший варіант вказує виконати асинхронно задачу із переданими аргументами. Запуск відбувається на розсуд реалізації – не обов'язково задача почне виконуватися зразу.
- Другий варіант дозволяє обрати політику запуску. Стандарт гарантує наявність принаймні двох політик: std::launch::async і std::launch::deferred. Реалізації можуть додавати свої.
- Перше перевантаження функції еквівалентне використанню у другому політики запуску "std::launch::async | std::launch::deferred" — вибір режиму надається на розсуд реалізації.

[45] В C++14, C++17, C++20 прототип дещо змінювався, детальніше див. http://en.cppreference.com/w/cpp/thread/async.

Політики async()

- Політика **std::launch::async** приводить до запуску задачі в окремому потоці, способом, еквівалентний до виклику конструктора ”std::thread(f, args...)”.
- Якщо негайний запуск в іншому потоці неможливий, буде кинути виключення std::system_error з кодом std::errc::resource_unavailable_try_again.
- Політика **std::launch::deferred** відтерміновує запуск до моменту, коли буде викликано get() чи wait() — в цей момент задача виконається синхронно.
  - Виклики wait_for() чи wait_until() для такої задачі повернуть future_status::deferred.
  - Виклик wait() дозволяє виконати обчислення до того, як із ними безпосередньо працюватимуть.
  - Якщо get() чи wait() не було викликано, то deferred-задача може не виконуватися взагалі.
  - Важливо, що deferred-задача може бути запущена не тільки в тому потоці, де її створили, але й в інших.

- Різним політикам відповідають різні біти відповідного аргументу. Тому ”std::launch::async | std::launch::deferred” означає – ”на вибір реалізації”.
- Реалізації можуть надавати свої політики. Приклад можливих політик: ”sync” — негайний, синхронний, запуск задачі.
- В цілому, запуск без явного вказання політики видається поганою ідеєю, через його непередбачуваність.
- ◇ Надавайте перевагу політику вказувати явно.

- Вимога ”запускати функцію так, ніби вона передана конструктору std::thread” ефективно вбила можливість використовувати thread pool чи щось аналогічне.
- Microsoft поривалася скористатися стандартним thread pool Windows, але через скарги – не завжди викликаються деструктори TLS-об’єктів, це було сприйнято як баг.
- Подробиці: `https://stackoverflow.com/questions/50898954/possible-stdasync-implementation-bug-windows`, `stackoverflow.com/questions/50897768/in-visual-studio-thread-local-variables-destructor-not-called-wh` `50898570`, `https://docs.microsoft.com/en-us/cpp/standard-library/future-functions?view=vs-2019`.

Блокування в деструкторі future<> із async

- Якщо future<> повернений із async() із політикою launch::async, не було переміщено чи використано для ініціалізації посилання, його деструктор блокуватиме до завершення задачі.
- Тому, зокрема, код такого виду, фактично, є синхронним:

```
std :: async( std :: launch :: async,  []{  f (); }});
```

- Причина – уникнення втрати контролю над потоком, у якому виконується так запущена функція. Подробиці див. `https://stackoverflow.com/questions/23455104/why-is-the-destructor-of-a-future-returned-from-stdasync-blockin`
- Додатково, аналіз від Скотта Меєрса, із обґрунтуванням, що це реалізовується спеціальним спільним станом а не спеціальним future: `http://scottmeyers.blogspot.com/2013/03/stdfutures-from-stdasync-arent-special.html`.
- **Деструктори інших future<> не блокують ніколи**.

Приклади типових проблем із async()

- std::async() — високорівневий засіб, простий у використанні. Точніше, не так простий, як із малим бар'єром входу.
- Однак, в поточному стані std::async() – доволі примітивний інструмент із обмеженою областю застосування.
- Якщо час виконання схожий на послідовний код, не зважаючи на чисельні async(), ймовірно, він не запустився паралельно. Причин може бути багато. Обрано політику deferred, реалізація не змогла чи не захотіла створювати потоки, тощо. Єдиного підходу на такі випадки немає, але можна почати із явного вказання політики std::launch::async.
- Через фактичну відсутність підтримки яких-небудь екзекуторів, керування використанням фізичних ресурсів доводиться здійснювати програмісту.

- Одна із типових помилок наступна. Нехай є дві функції, які можуть виконуватися одночасно, а результат – сума повернених ними значень. Вирішено одну запустити в поточному потоці, а іншу — за допомогою async():

```
future<int> result1 { async( f1 ) };

int complete_result = result1 .get() + f2 ();
```

Запуск цього коду може виявити, що жодного прискорення не відбулося. Може не виявити цього сумного факту. Справа в тому, що порядок обчислення аргументів функцій не фіксується. Тому може статися, result1.get() буде викликано першою, блокуючи поточний потік до завершення f1() і аж тоді надаючи нагоду працювати f2().

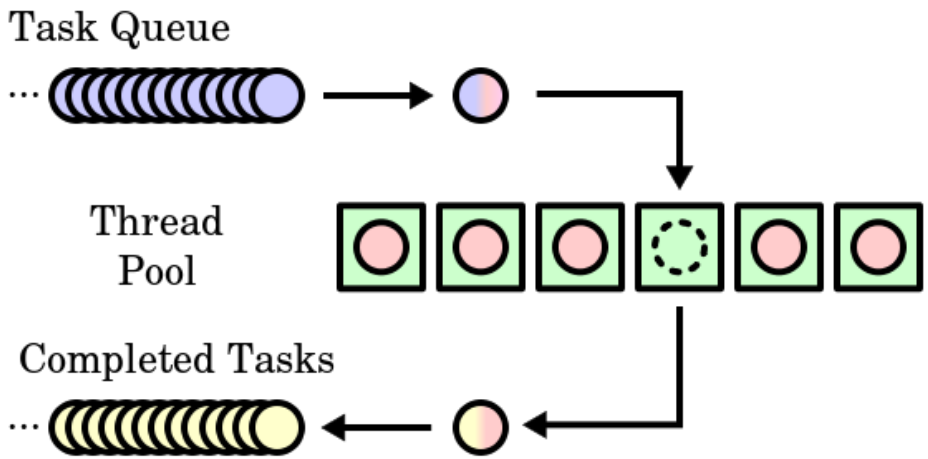
- Тому слід робити так:

```
future<int> result1 { async( f1 ) };

int complete_result = f2 ();
complete_result += result1 .get ();
```

- Взагалі, загальне правило асинхронного виконання повільних задач — запускати чим-раніше, питатися про результат чим-пізніше.

- Створення потоків – не безкоштовне.
- Тому широко використовується поняття, паттерн проектування, thread pool: існує множина наперед створених потоків, які беруть задачі із черги та виконують їх:



- При тому, кількість потоків, що очікують на задачі, їх диспетчеризація, організація черги задач тощо, можуть як встановлюватися статично, так і змінюватися динамічно, в залежності від навантаження.
- **Thread pool** – частковий випадок поняття Executor.
- **Executor** – об’єкт, що контролює, як, де і коли задача буде виконуватися.
- На жаль, стандартна бібліотека C++ не містить ніяких thread-pool подібних засобів.
- ◇ Спочатку була думка, що реалізація async() може на якісь такі засоби покладатися, але на практиці – ні, див. відповідну частину.

Найпростіша реалізація

- За відсутності готового, thread pool доводиться реалізовувати самостійно.
- Найпростіший приклад від Anthony Williams, із змінами:

```
class thread_pool{
    //! Порядок оголошень важливий! joiner має бути
    //! останнім, а threads -- передостаннім.
    //! Причина: знищення членів відбувається знизу вверх.
    std::atomic_bool done;
    threadsafe_queue< std::function<void()> > work_queue;
    std::vector<std::thread> threads;
    join_threads joiner;
    void worker_thread(); //<== magic is here
public:
    explicit thread_pool(size_t thr_n);
    ~thread_pool();

    template<typename FunctionType>
    void submit(FunctionType f){
        work_queue.push(std::function<void()>(f));
    }
};
```

```
thread_pool :: thread_pool( size_t  thr_n ):  done( false ),  joiner ( threads )  {
        try  {
            for( size_t  i=0;  i<thr_n;  ++i)  {
                threads .push_back(
                    std :: thread(&thread_pool :: worker_thread, this )    );
    // &thread_pool::worker_thread -- вказівник на член, який буде
    // викликано для обєкта' this
            }
        }
        catch (...)  {
            done=true;
            throw;
        }
    }

class  join_threads  {
    std :: vector<std :: thread>& threads;
public:
    explicit  join_threads ( std :: vector<std :: thread>& threads_):
        threads ( threads_ )  {}
    ~join_threads ()  {
        for( size_t  i=0;  i<threads . size ();  ++i)  {
            if ( threads [ i ]. joinable ())
                threads [ i ]. join ();
        }
    }
```

```
void thread_pool::worker_thread(){
    while(!done) {
        std::function<void()> task;
        if(work_queue.try_pop(task)) {
            task();
        }
        else {
            std::this_thread::yield();
        }
    }
}
thread_pool::~thread_pool(){
    done=true;
}
```

- Такий простий thread pool має багато проблем.
- В чому вони полягають та як їх вирішувати – див., наприклад главу 9 книги A. Williams-a ”C++ concurrency in action” та підходи, описані в ”Java concurrency in practice”.
- Зокрема, зверніть увагу на локальні черги та work stealing.

Windows thread-pool

- Windows кожній програмі створює thread pool (TP) за замовчуванням.
- Windows 10 використовує його для прискорення завантаження DLL, тому (станом на весну 2020), після запуску програми у цьому TP є 3 потоки, що очікують.
- Заборонити їх створення можна лише для конкретних exe-файлів, за допомогою реєстру. Подробиці див. `https://stackoverflow.com/questions/42789199/why-there-are-three-unexpected-worker-threads-when-a-win32-conso` `42789684`, `https://threatvector.cylance.com/en_us/home/windows-10-parallel-loading-breakdown.html`.
- Документація: `https://docs.microsoft.com/en-us/windows/win32/procthread/thread-pools`, `https://docs.microsoft.com/en-us/windows/win32/procthread/thread-pool-api`.
- Приклад C++ обгортки над стандартним thread pool Windows: `http://web.archive.org/web/20200323142733/https://dorodnic.com/blog/2015/10/17/windows-threadpool/`.
- "Thread pool threads are like preschool: Leave things the way you found them".

Посилання

- Ті чи інші екзекутори містить багато бібліотек: boost::thread, boost::asio, facebook folly, intel TBB, Qt тощо. Вони відрізняються рівнем абстракції, конфігурабельністю, зручністю тощо – вибирайте уважно.
- Крихітна heasers-only реалізація: `https://github.com/inkooboo/thread-pool-cpp`.

Parallel STL (C++17) (DRAFT)

- Parallelism TS, C++17 – паралельні версії алгоритмів: for_each, for_each_n, reduce, exclusive_scan, inclusive_scan, transform_reduce, transform_exclusive_scan, transform_inclusive_scan.
- Приклад – паралельний transform_reduce()[46].
  - Розглянемо одне із перевантажень:

```
template<class ExecutionPolicy, class ForwardIt, class T,
         class BinaryOp, class UnaryOp>
T transform_reduce(ExecutionPolicy&& policy,
                   ForwardIt first, ForwardIt last,
                   T init, BinaryOp binary_op, UnaryOp unary_op);
```

  - Execution policy визначає способи розпаралелення.
  - Функція unary_op() застосовується до кожного елементу вхідного інтервалу.
  - Потім, над її результатами виконується binary_op(), починаючи із значення init, у не фіксованому порядку – здійснюється редукція.

[46] Існує також послідовний аналог. Всього їх, станом на C++17/C++20 6 варіантів, див. https://en.cppreference.com/w/cpp/algorithm/transform_reduce.

- Приклад використання:

```
using dictionary_t = map<std:: string , size_t >;
 dictionary_t merge_dictionaries ( dictionary_t , const dictionary_t &);
 dictionary_t index_archive (const std :: string & filename );
        .....................
      auto words_counter = std :: transform_reduce (
                std :: execution :: par ,
                filenames .begin (), filenames .end (),
                 dictionary_t {}, // Type of result would be dictionary_t
                 merge_dictionaries ,
                index_archive
                );
```

- Детальніше, див., наприклад:
  `https://en.cppreference.com/w/cpp/algorithm`
- Підтримуються останніми версіями MSVC, останні версії GCC і Clang підтримують за наявності TBB 2018 або новішої[47].

[47] Тому на Ubuntu 2018 є проблема – там менеджер пакетів інсталює TBB 2017.

- Можливі політики (header <execution>)[48]:
  - std::execution::sequenced_policy seq – кроки алгоритму виконуються послідовно, у потоці, що викликав алгоритм, їх порядок не фіксується.
  - std::execution::parallel_policy par – може виконуватися в цьому потоці та в інших потоках, неявно створених кодом стандартної бібліотеки. Порядок не фіксується ні між потоками, ні в межах одного потоку.
  - std::execution::parallel_unsequenced_policy par_unseq – алгоритм може паралелізуватися з використанням потоків і/або векторизації. Також, задачі можуть мігрувати між потоками.
  - std::execution::unsequenced_policy unseq (C++20) – алгоритм може використовувати векторизацію для розпаралелення в межах поточного потоку.
  - Реалізації можуть додавати свої, типу parallel::cuda чи parallel::opencl.
  - Якщо при опрацюванні котрогось із елементів інтервалів, сталося виключення, для стандартних політик програма завершується std::terminate(). Однак, реалізації можуть надавати політики, які по іншому поводяться із виключеннями.

[48] https://en.cppreference.com/w/cpp/algorithm/execution_policy_tag та https://en.cppreference.com/w/cpp/algorithm/execution_policy_tag_t.

Потоки C11

- В C11 додано схожі інструменти підтримки багатопоточності. Наприклад:
- thrd_t — структура, що описує потік, та ряд функцій із очевидними назвами, наприклад thrd_create() thrd_join(), thrd_detach(), thrd_yield(), для роботи із ним.
- mtx_t — структура, що описує мютекс, та відповідні функції mtx_init(), mtx_lock(), mtx_unlock(), тощо.
- Підтримуються також умовні змінні: cnd_t та робота із ними; call_once() і once_flag; thread-local storage;
- Підтримка цих засобів компіляторами опціональна — якщо оголошено макрос `__STDC_NO_THREADS__`, вони відсутні.
- На загал, бібліотека відповідає підходу PThreads.
- Засоби типу future/promise/async() відсутні.
- Детальніше див. `http://en.cppreference.com/w/c/thread`.
- Восени 2023 про підтримку оголосила MSVC: `https://devblogs.microsoft.com/cppblog/c11-threads-in-visual-studio-2022-version-17-8-preview-2/`.

boost::thread

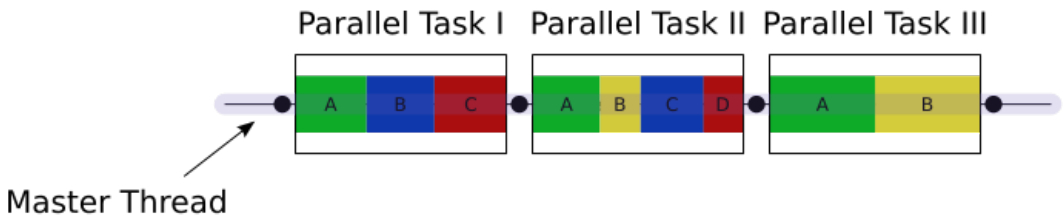
- boost містить ряд цікавих інструментів, відсутніх у стандарті.
- ```
  class barrier {
  public:
      barrier ( barrier const&) = delete ;
      barrier & operator=( barrier const&) = delete ;

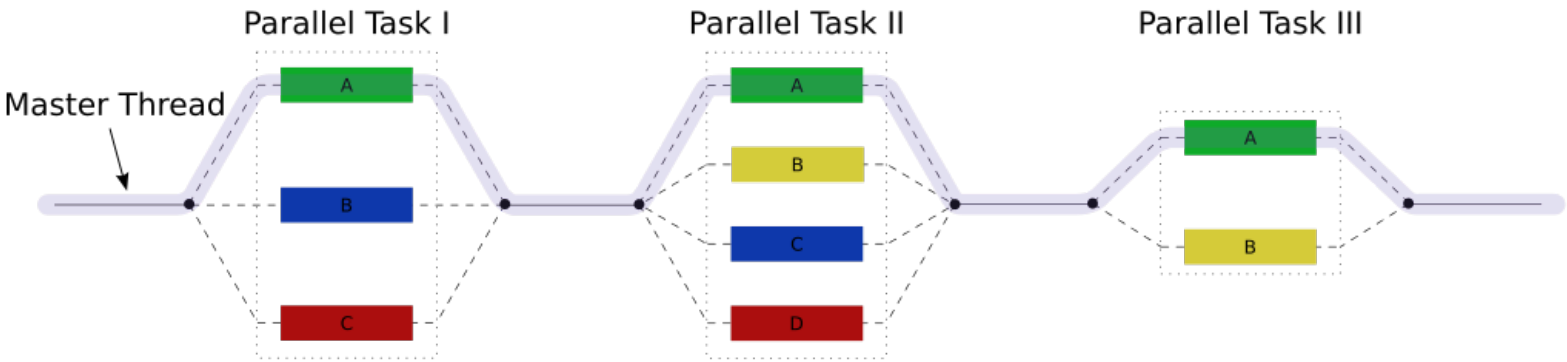
      barrier (unsigned int count );
      template <typename F>
      barrier (unsigned int count, F&&); // Call completion function

      ~ barrier ();

      bool wait ();
      void count_down_and_wait();
  };
  ```
- Експериментальний клас latch для координації потоків – свого роду гібрид бар'єра і семафору.
- Див. також `https://www.modernescpp.com/index.php/latches-and-barriers`.
  ```

- Boost fork-join framework:





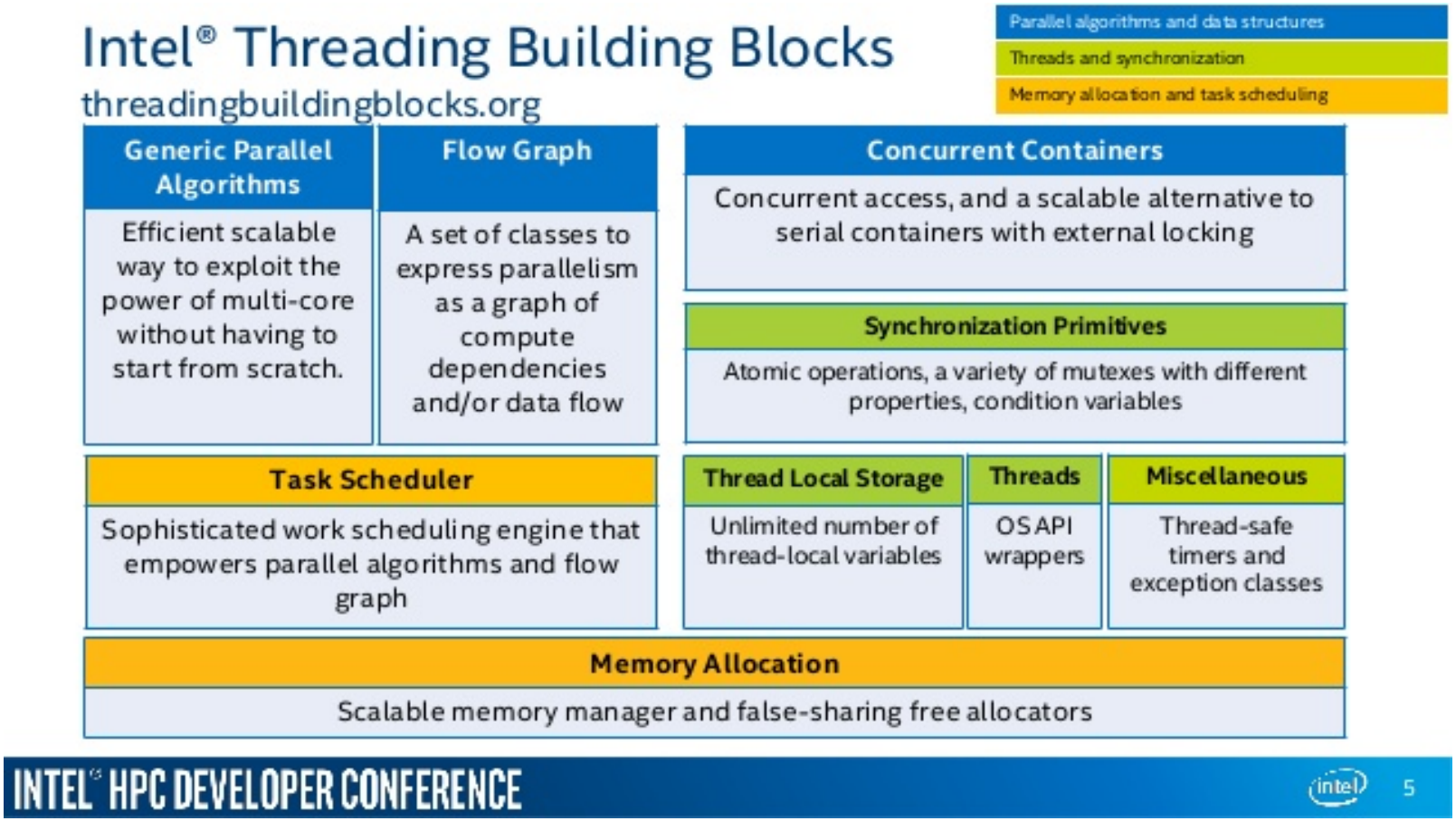
- Також boost містить дещо розширений варіант бібліотеки потоків, з якої було виділено та стандартизовано std::thread.

Intel Threading Building Blocks

- parallel_for, parallel_reduce, parallel_scan
- parallel_while, parallel_do, parallel_pipeline, parallel_sort
- concurrent_queue, concurrent_priority_queue, concurrent_vector, concurrent_hash_map
- scalable_malloc, scalable_free, scalable_realloc, scalable_calloc, scalable_allocator, cache_aligned_allocator
- mutex, spin_mutex, queuing_mutex, spin_rw_mutex, queuing_rw_mutex, recursive_mutex
- Та багато іншого – атомарні змінні та операції, керування диспетчеризацією задач тощо.

Intel Threading Building Blocks структура (DRAFT)

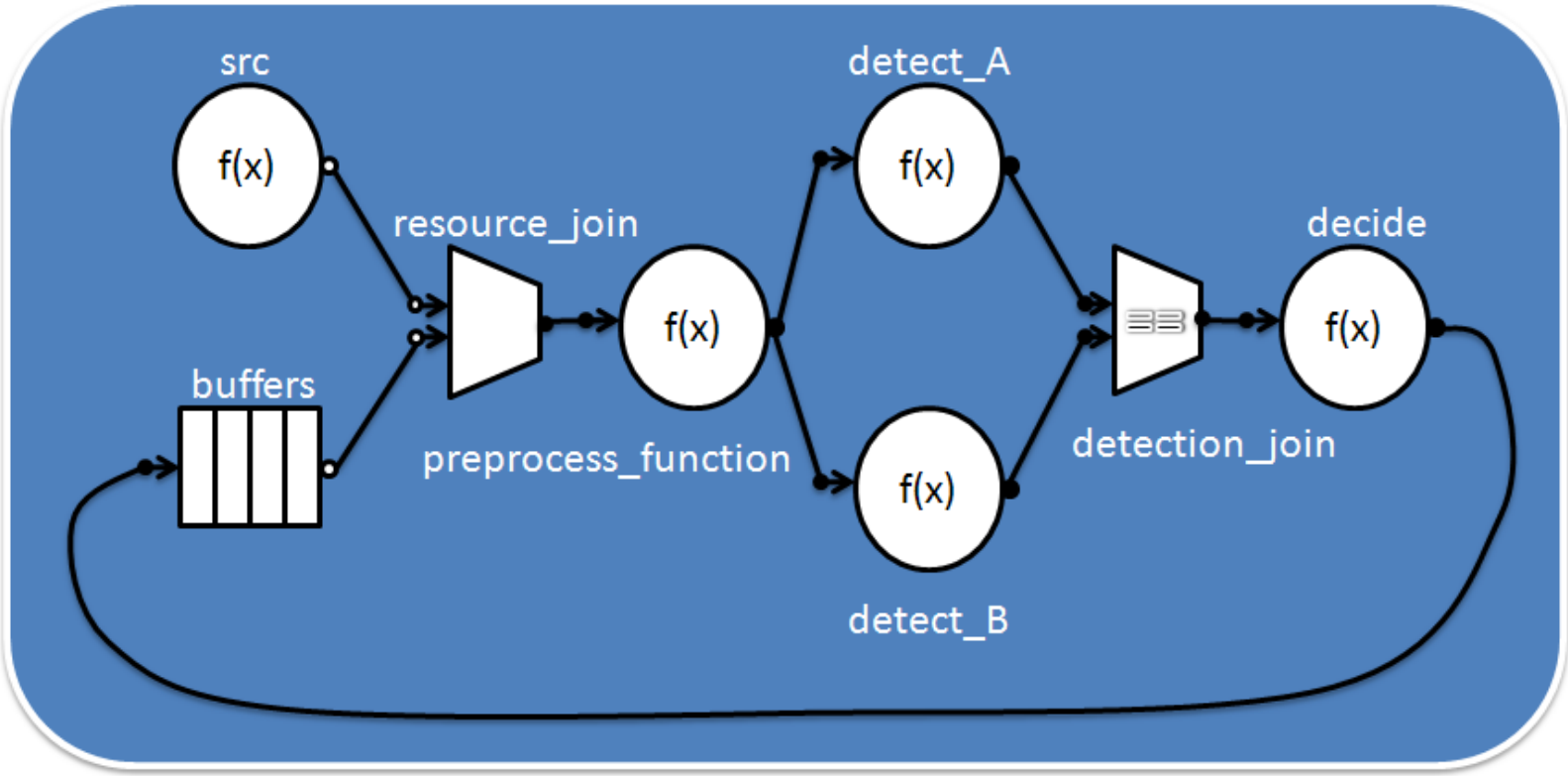
- Загальна структура:



- Взято із: `https://www.slideshare.net/IntelSoftware/use-c-and-intel-threading-building-blocks-intel-tbb-for-hardware`

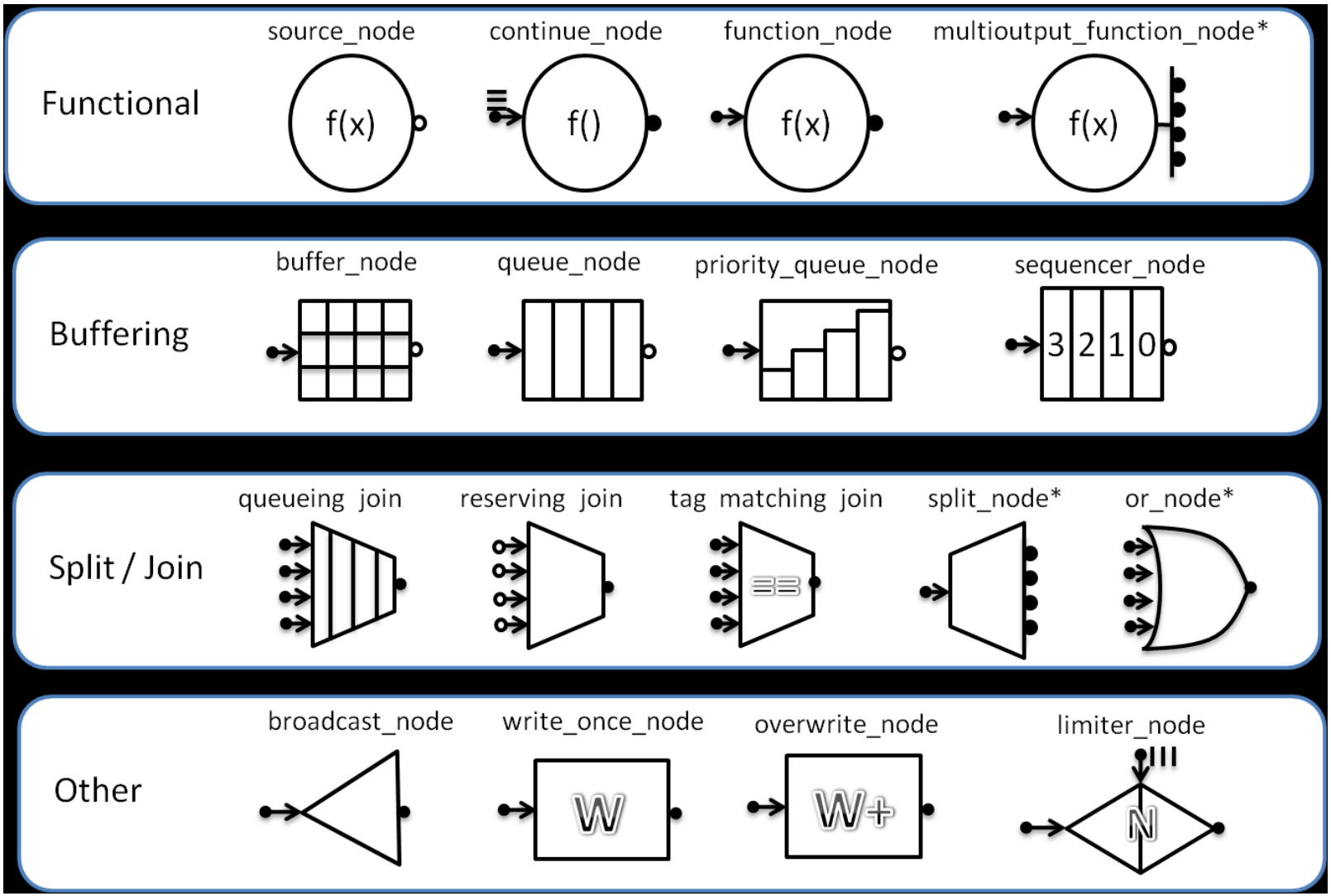
Intel Threading Building Blocks flow graph (DRAFT)

- Приклад flow graph:



Intel Threading Building Blocks flow graph (DRAFT)

- Види вузлів flow graph:



Intel Threading Building Blocks – посилання

- ”Pro TBB: C++ Parallel Programming with Threading Building Blocks”, Michael VossRafael AsenjoJames Reinders.
  - ◇ На даний момент доступна безкоштовно: `https://link.springer.com/book/10.1007/978-1-4842-4398-5`.
- `https://github.com/oneapi-src/oneTBB`[49]
- Документація: `https://software.intel.com/content/www/us/en/develop/documentation/tbb-tutorial/top.html` та `https://software.intel.com/content/www/us/en/develop/documentation/tbb-documentation/top.html`.
- `https://www.threadingbuildingblocks.org/learn`[50].
- `http://www.cs.cmu.edu/afs/cs/academic/class/15499-s09/www/handouts/TBB-HPCC07.pdf` – презентація-огляд.
- `http://www.physics.ntua.gr/~konstant/HetCluster/intel12.1/tbb/Design_Patterns.pdf` – Intel Threading Building Blocks Design Patterns.

[49] Якраз зараз відбувається перехід на/інтеграція із Intel oneAPI.

[50] Джерело цікаве, але на момент написання статус його незрозумілий. Сам домен перенаправляє на github із oneTBB.

Facebook Folly

- Внутрішня бібліотека C++ компонент Facebook (Open source).
- Кросплатформова лише умовно[51], хоча ситуація покращується.
- Поміж іншого[52] містить:
- Атомарні структури даних: AtomicHashMap, AtomicHashArray, AtomicHashArray, AtomicLinkedList, ...
- MPMCQueue, ConcurrentSkipList, TimeoutQueue
- Різноманітні засоби синхронізації (Baton, RWSpinLock, ScopeGuard, SmallLocks, бібліотеку Synchronized – ідейний розвиток монітору Java, synchronized блоків чи методів)
- Засоби вводу-виводу (аналог – boost::asio).
- Просунуті futures, зокрема із підтримкою continuation.
- Екзекутори: ThreadedExecutor, що кожного разу запускає новий потік, CPUThreadPoolExecutor та інші нащадки ThreadPoolExecutor – реалізації threadpool тощо.

[51] Якщо у вас не Ubuntu 2016 – шанси успішно скомпілювати самостійно не дуже високі, хоча є інструкції чи й бінарні пакети для інших версій Лінукс та інших ОС.

[52] Своїх реалізацій vector, string, бітових операцій тощо

Facebook Futures (DRAFT)

- Простий приклад:

```
bool process_result (int arg) { .... }

 folly :: ThreadedExecutor executor;
Promise<int> p;
Future<int> f = p.getSemiFuture(). via(&executor);
auto f2 = move(f).thenValue( process_result );

auto res = move(f2).get ();

// Another event callback/another thread:
p. setValue (42);
```

- Різні екзекутори для різних continuation:

```
std::move(aFuture)
  .thenValue(x)
  .via(e1).thenValue(y1).thenValue(y2)
  .via(e2).thenValue(z);
```

Без них, де виконуватиметься continuation – на загал, невідомо. Це може бути потік, що робить set, що робить get, чи той, що виконав .then.

- Всі continuation потрібно писати одним виразом, щоб не було гонитви:

```
f2 = std::move(f).via(e1).thenValue(y1).thenValue(y2); // nothing racy
std::move(f2).thenValue(y3);                           // racy
```

- Комбінування:

```
SomeAsyncProcessingTool mc;

vector<SemiFuture<GetReply>> futs;
for (auto& key : keys) {
    futs.push_back(mc.future_get(key));
}
auto all = folly::collectAll(futs.begin(), futs.end());
// Або:
auto any = folly::collectAny(futs.begin(), futs.end());
// Або:
auto reduced = folly::reduce(futs.begin(), futs.end(), init,
                             reducing_fn);
// Або:
auto reduced = folly::unorderedReduce(futs.begin(), futs.end(), init,
                                      reducing_fn);
```

```
folly::ThreadedExecutor executor;
SemiFuture<GetReply> semiFut = mc.future_get("foo");
Future<GetReply> fut1 = std::move(semiFut).via(&executor);

Future<string> fut2 = std::move(fut1).thenValue(
  [](GetReply reply) {
    if (reply.result == MemcacheClient::GetReply::Result::FOUND)
      return reply.value;
    throw SomeException("No value");
  });
Future<Unit> fut3 = std::move(fut2)
  .thenValue([](string str) { // Use value, passes error to next.
    cout << str << endl;
  })
  .thenTry([](folly::Try<string> strTry) { // Uses both value and error
              // folly::Try<T> encapsulates both value and exception.
    cout << strTry.value() << endl;
  })
  .thenError(folly::tag_t<std::exception>{},
              [](std::exception const& e)    // Users error, passes value.
  {                                          // tag_t allows filtering by exception type.
    cerr << e.what() << endl;
  });
```

- Готуємо результат:

```
Promise<int> p;
p.setValue(42);
// Або:
p.setException(std::runtime_error("Fail"));
```

- Або навіть так:

```
p.setWith([]{
  try {
    // do stuff that may throw
    return 42;
  } catch (MySpecialException const& e) {
    // handle it
    return 7;
  }
  // Any exceptions that we didn't catch, will be caught for us
});
```

- Іноді зручно створювати Future вже готовим:

```
return folly::makeFuture(res_type{});  чи

return folly::makeFuture(std::current_exception());
```

Приклад Callback hell

```
void asyncA(Output, Callback<OutputA>);
void asyncB(OutputA, Callback<OutputB>);
void asyncC(OutputB, Callback<OutputC>);
void asyncD(OutputC, Callback<OutputD>);

auto result = std::make_shared<double>();
fooAsync(input, [=](Output output) {
  // ...
  asyncA(output, [=](OutputA outputA) {
    // ...
    asyncB(outputA, [=](OutputB outputB) {
      // ...
      asyncC(outputB, [=](OutputC outputC) {
        // ...
        asyncD(outputC, [=](OutputD outputD) {
          *result = outputD * M_PI;
        });
      });
    });
  });
});

// As an exercise for the masochistic reader, express the same thing without
// lambdas. The result is called callback hell.
```

Без Callback hell – continuations

```
Future<OutputA> futureA(Output);
Future<OutputB> futureB(OutputA);
Future<OutputC> futureC(OutputB);
OutputD d(OutputC) {
   if (somethingExceptional) throw anException;
   return OutputD();
}

Future<double> fut =
   fooFuture(input)
   .then(futureA)
   .then(futureB)
   .then(futureC)
   .then(d)
   .then([](OutputD outputD) { // lambdas are ok too
      return outputD * M_PI;
   });
```

- Приклад взяв тут: ”Futures for C++11 at Facebook”.

Посилання

- `https://github.com/facebook/folly/blob/master/folly/docs/Futures.md` – офіційна документація.
- `https://github.com/facebook/folly` – репозиторій, разом із інструкціями по інсталяції[53].
- `https://engineering.fb.com/developer-tools/futures-for-c-11-at-facebook/` – огляд ідеї, 2015. API з того часу трохи змінилося, щоправда, але концепції – збереглися.
- Додатково, приклад простенького callback hell: `https://www.boost.org/doc/libs/1_73_0/doc/html/boost_asio/example/cpp11/echo/async_tcp_echo_server.cpp`.

[53] Якщо коротко: Ubuntu 16/18/20 працює нормально, інші дистрибутиви – мали б теж. MSVC – vcpkg підтримує. OS X теж підтримується. MSYS2/Mingw – поки проблема, може потребувати великих зусиль. Підключення за допомогою CMake потребує трохи магії.

Asynqro та Async++

- Asynqro – проста бібліотека, що реалізовує засоби типу Future і екзекуторів folly.
- `https://github.com/dkormalev/asynqro`.
- Дружить Qt (аж по метод fromQtFuture) та іншими бібліотеками.
- Async++ – ще одна, простіша, реалізація async із continuations: `https://github.com/Amanieu/asyncplusplus`.

Приклад із документації. API повертає Future.

```
Future<bool, MyFailure> Worker::fetchData(QString username, QString password)
{
  return api->authenticate(username, password).flatMap([this](const User &userInfo) {
    auto taken = api->fetchTakenBooks().flatMap(
     [this](const QVector<QString> &bookIds) {
      QVector<Future<Book>> result;
       result.reserve(bookIds.count());
      for (const QString &id : bookIds)
          result << api->fetchBook(id);
      return Future<Book>::sequence(result);
    })
    .map([this](const QVector<Book> &books) { return Book::qmled(books); })
    .onSuccess([this](const QVariantList &books) { emit loanedBooksFetched(books); });

    auto suggestions = api->fetchSuggestions().innerFilter(
          [userInfo](const Book &book) {
              return book->ageRestriction < userInfo.age;
    })
    .map([](const QVector<Book> &books) { return Book::qmled(books); })
    .onSuccess([this](const QVariantList &books) { emit suggestionsFetched(books); });

    return taken.zip(suggestions).andThenValue(true);
  });
}
```

YACLib – Yet Another Concurrency Library

- Статична C++ бібліотека паралельного програмування з великим набором інструментів.
- Включає різноманітні пули потоків та інші екзекутори, свої future з continuation, підтримку coroutines з C++20, включаючи м'ютекси для них та їх перекидання між екзекуторами тощо.
- `https://github.com/YACLib/YACLib`.

Приклади з документації

```
yaclib::FairThreadPool cpu_tp{/*threads=*/4};
yaclib::FairThreadPool io_tp{/*threads=*/1};

yaclib::Run(cpu_tp, [] {  // on cpu_tp
  return 42;
}).ThenInline([](int r) {  // called directly after 'return 42', without Submit to cpu_tp
  return r + 1;
}).Then([](int r) {  // on cpu_tp
  return std::to_string(r);
}).Detach(io_tp, [](std::string&& r) {  // on io_tp
  std::cout << "Pipeline result: <" << r << ">" << std::endl;  // 43
});
                .....
yaclib::FairThreadPool tp{/*threads=*/4};
std::vector<yaclib::Future<int>> fs;

for (std::size_t i = 0; i < 5; ++i) {  // Run parallel computations
  fs.push_back(yaclib::Run(tp, [i] {  // connect with one of the database shards
    return i;
  }));
}

// Will be ready when any future is ready
WhenAny(fs.begin(), fs.size()).Detach([](int i) { /* some work with database */ });
```

ParlayLib

- Бібліотека паралельного програмування з: sequence – паралельним аналогом вектора, hashtable – phase-concurrent hashtable[54], Delayed Sequence – для lazy evaluations, деякі паралельні алгоритми тощо.
- `https://cmuparlay.github.io/parlaylib/`

[54] Можна паралельно виконувати вставку, пошук, видалення – але тільки щось одне, не змішуючи

Інші посилання

- ”The Fastest Mutexes”
- ”Scheduling threads like Thomas Jefferson”
- ”LMAX Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads”

